SZEGEDI TUDOMÁNYEGYETEM

Informatikai Intézet

SZAKDOLGOZAT

Kerti Gábor Kelemen

2026

SZEGEDI TUDOMÁNYEGYETEM

Informatikai Intézet

Mesterséges intelligencia alapú receptgeneráló webalkalmazás

Szakdolgozat

Készítette:

Kerti Gábor Kelemen

programtervező informatikus BSc szakos hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi docens

Szeged

2026

Feladatkiírás

Tartalmi összefoglaló

Téma megnevezése:

Mesterséges intelligencia alapú receptgeneráló webalkalmazás

A megadott feladat megfogalmazása:

A feladat egy olyan webalkalmazás létrehozása, amely a felhasználóktól kapott hozzávalók alapján automatikusan, strukturált recepteket generál mesterséges intelligencia segítségével, továbbá lehetőséget biztosít a személyes receptgyűjtemények mentésére és visszakeresésére. A dolgozat célja annak vizsgálata, hogy a nulláról tanított Transformer modellek vagy az előtanított, finomhangolt modellek teljesítenek-e jobban strukturált receptgenerálási feladatban.

Megoldási mód:

A feladat megoldásához Python és TypeScript programozási nyelveket, az adatbázis-kezeléshez PostgreSQL-t, a szöveggeneráláshoz pedig egy alapoktól felépített Transformer hálózatot, egy finomhangolt Flan-T5 modellt, illetve a Google Gemini API-ját fogok használni.

Alkalmazott eszközök, módszerek:

A mesterséges intelligencia modellek betanítása és futtatása PyTorch és Hugging Face Transformers könyvtárakkal történik. A backend architektúrát aszinkron FastAPI keretrendszerrel és SQLAlchemy ORM-mel építem fel, ahol a biztonságos hitelesítést JWT és Argon2 jelszóhasítás biztosítja. A frontend fejlesztéshez React Router v7-et használtam.

Elért eredmények:

A kutatás és fejlesztés során sikeresen optimalizáltam a receptgenerálási stratégiákat, és az eredmények azt mutatják, hogy a transzfer tanulás jobb minőségű recepteket generált, mint az alapoktól tanított modell.

Kulcsszavak:

mesterséges intelligencia, természetes nyelvfeldolgozás, Transformer, FastAPI, React, receptgenerálás

Tartalomjegyzék

Feladatkiírás.....1
Tartalmi összefoglaló.....2
Tartalomjegyzék.....3
Bevezetés.....5
1. Irodalmi áttekintés.....7
1.1. A Sequence-to-Sequence modellek.....7
1.2. A Transformer architektúra megjelenése.....7
1.3. A T5 (Text-to-Text Transfer Transformer) modellcsalád.....8
1.4. Szöveggenerálási és dekódolási stratégiák.....8
1.5. A transzfer tanulás (Transfer Learning) paradigmája.....9
1.6. A szöveggenerálás kiértékelési metrikái.....9
1.7. A nyelvi modellek hallucinációja.....10
2. Adatfeldolgozás és Gépi Tanulási Architektúrák.....12
2.1. Adatgyűjtés és adatelőkészítés.....12
2.1.1. Adat-augmentáció és zajhozzáadás (Noise Injection).....12
2.2. A mesterséges intelligencia modellek felépítése és tanítása.....13
2.2.1. Saját fejlesztésű Transformer.....13
2.2.2. Előtanított modell finomhangolása (Flan-T5 Fine-Tuning).....14
2.3. Külső nyelvi modellek integrációja és Prompt Engineering.....14
2.3.1. Prompt engineering és AI biztonság.....14
2.3.2. Hibatűrő hálózati architektúra.....15
2.4. A modellek kiértékelésének módszertana.....17
2.4.1. Formátumkövetés.....17
2.4.2. Szemantikai és lexikális egyezés mérése.....17
2.4.3. Kreativitás és Hallucináció mérése.....18
3. Szoftvertechnológia és Webes Architektúra.....18
3.1. Kliensoldali felület és renderelés.....18
3.2. Szerveroldali architektúra (Backend) és Adatbázis.....19
3.3. Adatbázis-tervezés és Biztonságos Adatáramlás.....19
3.3.1. Entitás-Kapcsolat (ER) modell és Adatszerkezet.....20
3.3.2. Szekvenciális adatáramlás: A receptmentés folyamata.....21
3.4. Fejlesztési és üzemeltetési környezet.....22
4. Eredmények.....22
4.1. A webalkalmazás funkciói és felhasználói felülete.....23
4.1.1. Hitelesítés és kezdőképernyő.....23
4.1.2. Dinamikus adatbevitel és Modellválasztás.....24
4.1.3. Aszinkron generálás és Optimista Állapotkezelés.....24
4.1.4. Eredmények összehasonlítása és Perzisztencia.....25
4.2. A gépi tanulási modellek kiértékelése.....26
4.2.1. A saját fejlesztésű modell eredményei.....26
4.2.2. A finomhangolt Flan-T5 modell eredményei.....28
4.2.3. Összehasonlító konklúzió.....29

5. Összefoglalás **30**
5.1. Továbbfejlesztési lehetőségek **31**
6. Mesterséges intelligencia használata a fejlesztés során **32**
6.1. Alkalmazott eszközök és feladatkörök **32**
6.2. Validációs módszertan és kritikus hibakezelés **32**
6.3. Az emberi tényező: Területek AI-támogatás nélkül **33**
6.4. Tapasztalatok és jövőbeli tanulságok **33**
Irodalomjegyzék **34**
Köszönetnyilvánítás **35**
Nyilatkozat **36**
Mellékletek **37**

Bevezetés

A huszonegyedik század harmadik évtizedébe lépve a mesterséges intelligencia, és különösen a generatív nagy nyelvi modellek rohamos fejlődése jelentős hatással volt a technológia használatára. Az OpenAI ChatGPT, a Google Gemini, vagy a Meta Llama modelljeinek megjelenése megmutatta, hogy a természetes nyelvfeldolgozás túllépett a klasszikus fordítási és szövegelemzési feladatokon. A mai modern rendszerek képesek emberi szintű, komplex és kreatív szöveggenerálásra, ami számos iparágban új alkalmazási területeket nyitott meg.

Ezzel a technológiai forradalommal párhuzamosan a mindennapi élet egyik legősibb területe, a gasztronómia és a háztartásvezetés is egyre inkább digitalizálódik. A modern, rohanó életmód gyakran állítja kihívás elé az embereket a napi étkezések megtervezésében. Egy átlagos háztartásban gyakori probléma, hogy a hűtőszekrényben csupán néhány látszólag össze nem illő, maradék alapanyag található. Az ezekből történő tudatos ételkészítés nem csupán időt és pénzt takarít meg a felhasználóknak, hanem globális szinten is kritikus szerepet játszik a fenntarthatóságban és az élelmiszer-pazarlás csökkentésében. Bár az interneten több millió recept érhető el, a hagyományos keresőmotorok és receptoldalak statikusak: megkövetelik, hogy a felhasználó pontosan tudja, mit akar főzni, és ahhoz vásároljon be. Egy olyan rendszer fejlesztése, amely a megfordított logikára épül, azaz a meglévő, véletlenszerű hozzávalókból generál egy új receptet, releváns szoftvermérnöki feladat.

A jelen szakdolgozat célja ezen kihívás szoftvermérnöki és adatkutatói megválaszolása: egy olyan webalapú receptgeneráló alkalmazás tervezése és fejlesztése, amely a felhasználó által megadott hozzávalók alapján, mesterséges intelligencia segítségével hoz létre egyedi, értelmezhető és strukturált recepteket.

A projekt megvalósítása egy kettős mérnöki kihívást foglal magába. Egyrészt egy stabil, moduláris architektúra felépítését igényli, amely a modern felhasználói élményt ötvözi a backend rendszerekkel. Másrészt a legmodernebb Deep Learning architektúrák vizsgálatát és integrációját teszi szükségessé. A kulináris szöveggenerálás speciális problémát jelent az NLP területén: egy jó receptnek nem csupán nyelvtanilag kell helyesnek lennie, hanem a főzés logikáját is követnie kell, elkerülve az emberi fogyasztásra alkalmatlan vagy értelmetlen hallucinációkat. Ennek vizsgálatára a dolgozat egy átfogó összehasonlító elemzést végez a klasszikus, nulláról tanított

neurális hálózatok és az előtanított, majd finomhangolt transzfertanulási architektúrák teljesítménye között.

A dolgozatban először ismertetem a természetes nyelvfeldolgozás és a szöveggeneráló hálózatok elméleti hátterét, illetve az alkalmazott dekódolási stratégiákat. Ezután bemutatom a mesterséges intelligencia modellek felépítését és betanítását, valamint az ezeket kiszolgáló webalkalmazás szoftveres és hálózati megtervezését. Az utolsó részben bemutatom, hogyan néz ki és hogyan működik a gyakorlatban az elkészült felhasználói felület, végül objektív metrikák alapján kiértékelem a modellek teljesítményét és összegzem a kutatás tapasztalatait.

1. Irodalmi áttekintés

A jelen fejezet a természetes nyelvfeldolgozás és a szöveggeneráló mesterséges intelligencia modellek elméleti hátterét, fejlődési ívét, valamint a dolgozatban alkalmazott dekódolási stratégiákat mutatja be a releváns szakirodalom alapján.

1.1. A Sequence-to-Sequence modellek

A természetes nyelvfeldolgozás történetében mérföldkőnek számítottak a kódoló-dekódoló (Encoder-Decoder) architektúrára épülő Sequence-to-Sequence (Seq2Seq) modellek megjelenése [1]. Ezek a hálózatok arra lettek tervezve, hogy egy tetszőleges hosszúságú bemeneti szekvenciát egy fix méretű kontextusvektorrá tömörítsenek, majd ebből egy szintén tetszőleges hosszúságú kimeneti szekvenciát generáljanak. Kezdetben ezek a modellek Rekurrens Neurális Hálózatokra (RNN), illetve Hosszú Rövidtávú Memória (LSTM) cellákra épültek. Bár a Seq2Seq modellek forradalmasították a gépi fordítást, az RNN-alapú architektúrák komoly korlátokkal küzdöttek: a szekvenciális adatfeldolgozás miatt nem voltak párhuzamosíthatók a betanítás során, illetve a hosszú szövegek esetén az "elfelejtés" problémájába ütköztek, mivel a teljes bemenetet egyetlen fix méretű vektorba kellett sűríteniük.

1.2. A Transformer architektúra megjelenése

Az NLP területén a legjelentősebb áttörést a Google kutatói által 2017-ben publikált "Attention Is All You Need" című tanulmány hozta el [2], amely bemutatta a Transformer architektúrát. A Transformer teljesen elhagyta a rekurzív és konvolúciós rétegeket, és helyettük kizárólag a figyelem-mechanizmusra (Self-Attention) támaszkodott.

A Self-Attention mechanizmus lehetővé teszi, hogy a modell a szöveg generálása során a bemeneti szekvencia minden tokenjét egyszerre, párhuzamosan vizsgálja meg, és matematikailag súlyozza, hogy egy adott szó szempontjából mely más szavak hordozzák a legfontosabb kontextust. Ez nemcsak a hosszú távú összefüggések hibátlan megértését tette lehetővé, hanem a hálózatok tanítását is hatékonnyá és hardveresen párhuzamosíthatóvá tette. A Transformer architektúra képezi az alapját a

napjainkban ismert összes modern nagy nyelvi modellnek, beleértve a GPT, a BERT és a T5 hálózatokat is.

1.3. A T5 (Text-to-Text Transfer Transformer) modellcsalád

Míg a korai Transformer modellek gyakran specializálódtak egy-egy feladatra (például a BERT a szövegértelmezésre, a GPT a generálásra), a Google 2020-ban bemutatta a T5 (Text-to-Text Transfer Transformer) modellcsaládot [3]. A T5 mögött meghúzódó legfőbb innováció az volt, hogy minden elképzelhető NLP problémát (fordítás, összefoglalás, kérdés-válasz, vagy esetünkben strukturált receptgenerálás) egy egységes text-to-text formátumra alakított át.

Ez a keretrendszer lehetővé teszi, hogy a modellt ugyanazzal a célfüggvénnyel és tanítási eljárással lehessen finomhangolni szinte bármilyen egyedi feladatra. A jelen dolgozat kísérleteihez használt Flan-T5 változat [4] egy olyan továbbfejlesztett verzió, amelyet instrukciók követésére (instruction fine-tuning) optimalizáltak, így magas pontossággal képes tartani az elvárt kimeneti struktúrákat.

1.4. Szöveggenerálási és dekódolási stratégiák

A Transformer modellek önmagukban csak valószínűség-eloszlásokat (logits) számolnak a szótárban lévő tokenekre vonatkozóan. Annak meghatározása, hogy ezekből a valószínűségekből hogyan lesz tényleges, olvasható szöveg, a dekódolási stratégián (Decoding Strategy) múlik [5]. A dolgozat kutatási szempontjából három fő stratégia elemzése elengedhetetlen:

A legegyszerűbb megközelítés a mohó keresés (greedy search), amely minden lépésben szigorúan a legmagasabb valószínűségű tokent választja. Bár gyors, gyakran vezet természetellenes, ismétlődő, vagy lokális optimumban ragadt szövegekhez.

A nyaláb keresés (beam search) a greedy search továbbfejlesztése, amely nem csak egy, hanem N darab legvalószínűbb útvonalat tart számon a generálás során, és a folyamat végén a globálisan legmagasabb valószínűségű szekvenciát választja. Ez a módszer koherens és logikus szöveget eredményez, azonban kreatív tartalom gyártásnál gyakran sablonos eredményt adhat.

A mintavételezés (sampling) és a top-p dekódolás egy olyan megközelítés, amely nem determinisztikusan a legvalószínűbb szót választja, hanem a valószínűségek

alapján sorsol. A top-p dekódolás garantálja, hogy a modell kizárólag a legvalószínűbb szavak szűkített halmazából válasszon, ahol a szavak halmozott valószínűsége eléri a megadott p értéket. Ez a mechanizmus hatékonyan szűri ki a logikátlan és értelmetlen folytatásokat. A generálás sokszínűségét a hőmérséklet (temperature) paraméter finomhangolása egészíti ki, amely a kimeneti valószínűségek újraárazásával közvetlenül szabályozza a kreativitást. Magasabb érték esetén a hálózat bátrabban nyúl a ritkább, de stilisztikailag érdekesebb kifejezésekhez, ezáltal képes például váratlan vagy különleges fűszereket is belevonni a receptbe.

1.5. A transzfer tanulás (Transfer Learning) paradigmája

A természetes nyelvfeldolgozás korai szakaszában a specifikus feladatokra (például fordításra) szánt neurális hálózatokat jellemzően az alapoktól, véletlenszerűen inicializált súlyokkal kezdték el tanítani. Ez a megközelítés azonban adat- és számítási kapacitás-igényes, ráadásul a modellnek a célfeladat elsajátítása mellett magát a nyelvtant és a szavak alapvető jelentését is nulláról kell megtanulnia.

A modern NLP területén ezt a módszert felváltotta a transzfertanulás paradigmája [6]. Ennek lényege a "Pre-train and Fine-tune" stratégia. A folyamat első lépésében egy Transformer modellt hatalmas mennyiségű, általános szöveges adatokon tanítanak be. Ennek során a modell megtanulja a nyelv szintaktikai mintázatait, valamint a szavak kontextusfüggő jelentésreprezentációit, és általános nyelvi mintázatokat sajátít el a tanító adathalmazból. A második fázisban ezt az előtanított modellt egy sokkal kisebb, specifikus adathalmazon tanítják tovább. A transzfer tanulás hatékonysága abban rejlik, hogy a modellnek már nem a nyelvet kell megtanulnia, csupán a meglévő tudását kell adaptálnia az új feladat elvárt formátumához és logikájához, ami nagyságrendekkel jobb eredményeket produkál a nulláról tanított hálózatokkal szemben.

1.6. A szöveggenerálás kiértékelési metrikái

A generatív modellek teljesítményének objektív mérése az egyik legnagyobb kihívás a természetes nyelvfeldolgozás területén. Míg klasszikus osztályozási feladatok esetén egy predikció helyessége egyértelműen meghatározható, addig szöveggenerálási feladatoknál egy adott bemenetre több, egymástól eltérő, mégis nyelvtanilag és

szemantikailag helyes kimenet is létezhet. Emiatt a szigorú karakter- vagy szóegyezésen alapuló hagyományos metrikák korlátozottan alkalmazhatók.

A szakirodalomban ezért olyan mérőszámok terjedtek el, amelyek a generált és a referencia szövegek közötti hasonlóságot rugalmasabban kezelik. A ROUGE metrika [7] az egyik legszélesebb körben alkalmazott megközelítés, amely a két szöveg közötti átfedést vizsgálja az n-gramok szintjén. A különböző változatai eltérő aspektusokat mérnek: a ROUGE-1 az egyedi szavak egyezésére koncentrál, míg a ROUGE-L a leghosszabb közös részszekvencia alapján értékeli a hasonlóságot. Bár a módszer hatékony a lexikális átfedések kimutatásában, nem képes kezelni a jelentésben hasonló, de eltérő megfogalmazásokat.

Ezt a korlátot részben a BERTScore metrika [8] hidalja át, amely a szövegek összehasonlítását nem közvetlen szóegyezés, hanem kontextuális jelentésreprezentációk alapján végzi. A módszer előtanított nyelvi modellek segítségével vektoros beágyazásokat rendel a tokenekhez, majd ezek hasonlóságát méri, jellemzően koszinusz-távolság alapján. Ennek eredményeként a BERTScore képes figyelembe venni a szemantikai egyezéseket is, így olyan esetekben is magasabb értéket adhat, amikor a két szöveg eltérő szavakat használ, de jelentésük közel áll egymáshoz.

1.7. A nyelvi modellek hallucinációja

A nagy nyelvi modellek egyik legismertebb és legintenzívebben kutatott problémája a "hallucináció" [9]. Hallucinációnak nevezzük azt a jelenséget, amikor a modell folyékony, nyelvtanilag helyes, és látszólag logikus szöveget generál, amely azonban tényszerűen téves, vagy nem következik a megadott bemeneti adatokból. A Transformer modellek alapvetően valószínűség alapú gépek, amelyek a statisztikailag leginkább odaillő következő szót jósolják meg, valós tényellenőrző mechanizmus nélkül.

Egy strukturált receptgenerátor esetében a hallucináció tipikus megnyilvánulása, amikor a modell a bemenetként megadott hozzávalókon (például csirke, rizs) túl olyan alapanyagokat is beleír a receptbe (például tejszín vagy eper), amelyeket a felhasználó nem kért. Bár orvosi vagy jogi alkalmazásoknál a hallucináció kritikus hiba, a kreatív tartalomgyártás területén ez a jelenség bizonyos mértékig a kreativitás (a hiányzó elemek logikus pótlásának) jele is lehet. Ennek ellenőrzésére és kordában tartására a kutatások gyakran alkalmazzák az információ-visszakeresésből (Information Retrieval) ismert Precision (Precizitás) és Recall (Visszahívás) egyedi, tartalom-specifikus

változatait, amelyekkel számszerűsíthető a modell bemenethez való hűsége és kitaláló hajlama.

2. Adatfeldolgozás és Gépi Tanulási Architektúrák

A projekt megvalósítása során a fejlesztési folyamatot több, logikailag elkülönülő fázisra bontottam: az adatok összegyűjtésére és előkészítésére, a mesterséges intelligencia modellek betanítására, valamint az ezeket kiszolgáló elosztott webes architektúra kialakítására. Jelen fejezet ezeket a technológiai lépéseket és mérnöki döntéseket mutatja be.

2.1. Adatgyűjtés és adatelőkészítés

A nyelvi modellek betanításához a Kaggle platformon publikusan elérhető "Food.com Recipes and User Interactions" [10] adathalmaz RAW_recipes.csv fájlját alkalmaztam. Ez a nyers adathalmaz több százezer receptet tartalmaz, és olyan kulcsfontosságú metaadatokat foglal magába, mint a recept neve, elkészítési ideje, a hozzávalók listája és a lépések leírása.

A nyers adatok strukturálatlan formátuma miatt szigorú tisztítási folyamatra volt szükség. A hiányzó értékeket tartalmazó sorok eltávolítása után a karakterláncként (string) tárolt listákat a Python ast.literal_eval függvényével dolgoztam fel. A minőségi tanítás érdekében kiszűrtem a túlzottan rövid (4 lépésnél vagy 4 hozzávalónál kevesebb) és a túlzottan hosszú (20 lépés feletti) recepteket.

2.1.1. Adat-augmentáció és zajhozzáadás (Noise Injection)

A gépi tanulási modellek stabilitásának növelése és a "szótár-memorizálás" elkerülése érdekében egy egyedi adat-augmentációs eljárást dolgoztam ki. A tanítási adatok generálásakor a modell nem a tökéletes, eredeti hozzávaló-listát kapta meg bemenetként. A kód minden recept esetében véletlenszerűen eltávolított egy-két valós hozzávalót, miközben egy-két teljesen véletlenszerű, a recepthez nem tartozó, de az adathalmaz szótárában szereplő "zaj-elemet" adott hozzá. Ez a módszertani megközelítés rákényszeríti a neurális hálózatot arra, hogy ne csupán lemásolja a bemenetet, hanem kulináris logikát tanuljon: ismerje fel az oda nem illő elemeket, és logikusan következtesse ki a hiányzó alapanyagokat. A kimenetet egy szigorúan kötött formára állítottam: ingredients: ... title: ... time: ... steps: ...

2.2. A mesterséges intelligencia modellek felépítése és tanítása

A kutatás során két eltérő megközelítést alkalmaztam a szöveggenerálásra, amelyeket a PyTorch és a Hugging Face keretrendszerek segítségével implementáltam. A magas számítási kapacitást igénylő tanítási és finomhangolási folyamatokat a Google Colab Pro felhőalapú fejlesztőkörnyezetben hajtottam végre, ahol a hardveres gyorsítást és a nagyméretű tenzorműveletek aszinkron végrehajtását egy dedikált NVIDIA L4 GPU biztosította. A modellek teljesítményének nyomon követését és a betanítási metrikák naplózását közvetlenül a fejlesztőkörnyezetben végrehajtott egyedi szkriptekkel biztosítottam.

2.2.1. Saját fejlesztésű Transformer

Az alapoktól történő tanítás vizsgálatához egy egyedi, aszimmetrikus Transformer architektúrát építettem fel PyTorch-ban. A modell egy 12 000 szavas szótárral dolgozik, amelyet egy Byte-Level BPE tokenizálóval állítottam elő. Ez a viszonylag kompakt szótárméret a kulináris szövegek szűkebb, specifikus szókincséhez optimalizálva memóriahatékony és gyors tanítást tett lehetővé. Az architektúra beágyazási dimenziója 512, míg a hálózat 8 kódoló (encoder) és csupán 4 dekódoló (decoder) réteget tartalmaz. Ez az aszimmetrikus felépítés tudatos mérnöki döntés volt: jelentősen felgyorsítja a szavankénti (autoregresszív) szöveggenerálási fázist, miközben a kódoló rétegek túlsúlya révén megőrzi a hálózat komplex bemenet-értelmező (szemantikai) kapacitását. A rétegek 8 figyelem-fejjel (attention head) és a túltanulás (overfitting) megelőzése érdekében 10 százalékos Dropout rátával működtek, maximum 800 token hosszúságú szekvenciákon.

A betanításhoz a 150 000 receptből álló mintát a PyTorch WeightedRandomSampler osztályával súlyoztam, ezzel kezelve a bemeneti halmaz adat-kiegyensúlyozatlanságát, így a ritkább, hosszú és összetett receptek is reprezentatív arányban jelentek meg a tanítási batch-ekben. A hálózat optimalizálását a súlycsökkenést (weight decay) hatékonyabban kezelő AdamW algoritmussal végeztem. A konvergencia és a lokális minimumok elkerülése érdekében a tanulási rátát egy Cosine Annealing ütemező szabályozta, amely egy 5 százalékos bemelegítési fázis (warmup) után érte el az 5e-4 maximális értéket. A veszteségfüggvényként szolgáló CrossEntropyLoss-t 0.1-es címkesimítással (Label Smoothing) egészítettem ki; ez a

technika a modell predikciókba vetett túlzott magabiztosságának (overconfidence) büntetésével jelentősen csökkentette a generálás során fellépő hurkolódási és szóismétlési hibákat. A hálózat betanítása 20 epochon keresztül, 32-es batch mérettel történt. A végső szöveggeneráláshoz egy saját fejlesztésű intelligens dekódoló (Smart Decoder) algoritmust implementáltam, amely a Temperature (0.7) és a Top-K (20) mintavételezési stratégiákat ötvözve biztosítja a kimenet logikus, de kulinárisan kellően kreatív jellegét.

2.2.2. Előtanított modell finomhangolása (Flan-T5 Fine-Tuning)

A transzfertanulás vizsgálatához a Google által fejlesztett flan-t5-base modellt finomhangoltam. Az előtanított modell alkalmazása lehetővé tette, hogy a tanítást egy 30 000 elemből álló, szűkített receptadathalmazon végezzem el.

A tokenizálást követően a tanítás a Hugging Face Seq2SeqTrainer osztályának segítségével történt. A rendelkezésre álló hardverkorlátok figyelembevételével az alap batch méretet 8-ban határoztam meg, amelyet gradiens-akkumuláció alkalmazásával 16-os effektív batch méretre növeltem. A tanulási ráta értékét 3e-4-ben állítottam be, a tanítást pedig összesen 4 epochon keresztül végeztem.

A modell kiértékelése minden epoch végén megtörtént. Ennek során egy egyedi compute_metrics függvény számította ki és naplózta a ROUGE-1, ROUGE-2 és ROUGE-L metrikákat a generált és a referencia szövegek összehasonlítása alapján.

2.3. Külső nyelvi modellek integrációja és Prompt Engineering

A saját fejlesztésű és finomhangolt modellek mellett a webalkalmazás lehetőséget biztosít a Google Gemini nagy nyelvi modelljének meghívására is. Az API alapú integrációnak kettős célja volt: egyrészt egy modern, sokak által használt referenciapontot biztosít a saját modellek teljesítményének összehasonlításához, másrészt garantálja az alternatív receptgenerálást.

2.3.1. Prompt engineering és AI biztonság

Mivel a Gemini alapértelmezetten egy általános célú társalgási modell, amely hajlamos a bőbeszédű, formázatlan válaszadásra, a szoftveres parser számára elengedhetetlen

strukturált adatkimenet eléréséhez specifikus prompt engineering technikákat alkalmaztam. A backend a kérés összeállításakor egy szigorúan megfogalmazott, Zero-Shot alapú utasításkészletet küld a hálózatnak. Ez a prompt egzakt módon kikényszeríti, hogy a modell kizárólag egy érvényes JSON objektumot adjon vissza a megkövetelt kulcsokkal (title, time, ingredients, steps), szigorúan mellőzve a Markdown formázást, a felsorolásjeleket és a generatív modellekre jellemző udvarias kiegészítő szövegeket.

Mérnöki és biztonsági döntés volt az AI Biztonság integrálása a prompt szintjén. A szabad szöveges hozzávaló-bevitelből fakadóan a modell instrukciót kapott a mérgező vagy emberi fogyasztásra alkalmatlan anyagok szigorú kiszűrésére, miközben rugalmasságot is biztosít a hiányzó, de logikus alapanyagok (pl. só, olaj, víz) hozzáadására.

2.3.2. Hibatűrő hálózati architektúra

A külső felhőszolgáltatásokkal való kommunikáció során a hálózati stabilitás és az átmeneti szolgáltatáskiesések kezelése kritikus szempont. A kiesésekből eredő hibákat egy exponenciális visszalépésen (Exponential Backoff) alapuló hálózati architektúra kezeli. A modern felhőalkalmazások esetében (különösen a nagy nyelvi modellek hívásakor) gyakori problémát jelent a szolgáltatás túlterheltsége (Rate Limiting). Ennek kiküszöbölésére a backend oldali integráció nem fut rögtön hibára egy sikertelen API kérés esetén, hanem egy try-except blokkba ágyazva, ciklusosan próbálkozik újra. A szoftveres megoldás lényege (ahogyan azt az 1. kódrészlet is mutatja), hogy a várakozási idő nem konstans, hanem minden egyes meghiúsult kísérlet után exponenciálisan növekszik ($2n-1$ másodperc logikával). Ez a módszertan nem csupán a szerver stabilitását növeli, hanem tehermentesíti a hálózatot is: egy esetleges szolgáltatáskiesés esetén nem árasztja el azonnali újrakérésekkel a célállomást. A maximális várakozási idő (5 másodperc) garantálja, hogy a kliensoldali felület (Frontend) ne fagyjon be beláthatatlan ideig a válaszadás hiánya miatt.

A biztonságos szoftverfejlesztési elvek betartása érdekében a hitelesítéshez szükséges API kulcsokat a forráskódtól szigorúan elkülönítve, szerveroldali környezeti változókként (Environment Variables) olvassa be a rendszer. A visszakapott nyers válaszból egy dedikált extrakciós függvény nyeri ki és validálja a JSON struktúrát,

garantálva, hogy a frontend kliens minden esetben csak feldolgozható és biztonságos adatot kapjon meg.

```
last_error = None
for attempt in range(1, ML_MODEL_MAX_RETRIES + 1):
    try:
        response =
client.models.generate_content(model=model_name,
contents=prompt)
        raw_text = str(getattr(response, "text", "") or
"").strip()

        if not raw_text:
            raise RuntimeError("Gemini returned an empty
response.")

        return self._extract_and_process(raw_text,
ingredients)
    except Exception as e:
        last_error = e
        logger.warning(f"Attempt
{attempt}/{ML_MODEL_MAX_RETRIES} failed: {e}")
        if attempt < ML_MODEL_MAX_RETRIES:
            time.sleep(min(2 ** (attempt - 1), 5))
raise RuntimeError("Failed to connect after multiple
attempts.") from last_error
```

2.1. kódrészlet: Az exponenciális visszalépés implementációja pythonban

2.4. A modellek kiértékelésének módszertana

A betanított modellek objektív összehasonlítása és teljesítményük mérése céljából egyedi tesztelő és kiértékelő szkripteket készítettem Python nyelven, amelyeket szintén a Google Colab környezetben futtattam.

2.4.1. Formátumkövetés

A szoftveres integrálhatóság alapfeltétele, hogy a weblap frontendje megfelelően fel tudja dolgozni a modell válaszait. Ennek ellenőrzésére a kiértékelő szkriptben egy egyedi szövegfeldolgozó algoritmust implementáltam, amely megkísérelte a generált nyers szövegeket strukturált adatszótárakká bontani az ingredients:, title:, time: és steps:

kulcsszavak mentén. A Formátumkövetés metrika azt mérte, hogy a modellek hány esetben tartották be maradéktalanul ezt a szigorú szintaktikai elvárást anélkül, hogy a parser hibára futott volna.

2.4.2. Szemantikai és lexikális egyezés mérése

A generált receptek és az eredeti, adathalmazban található referenciareceptek összehasonlítására a szkriptek a ROUGE-1 és a BERTScore metrikákat számították ki a Hugging Face evaluate könyvtárának bevonásával. Tudatos mérnöki döntés volt a szigorúbb, szórendet és hosszabb szekvenciákat vizsgáló metrikák (mint a ROUGE-2 vagy a ROUGE-L) mellőzése. Mivel a receptgenerálás egy flexibilis, kreatív feladat, a pontos szórendi egyezés megkövetelése fals negatív (indokolatlanul alacsony) pontszámokhoz vezetett volna. A lexikális kulcsszó-átfedéseket és a konyhai szókincs meglétét a ROUGE-1 hatékonyan mérte, míg a mondatok mögöttes értelmének, szinonimáinak és a kulináris lépések tényleges jelentésének vizsgálatát a modern BERTScore algoritmus vette át. A BERTScore a tokenek kontextuális beágyazásának (embedding) koszinusz-hasonlóságát elemezte, számszerűsítve a szövegek szemantikai egyezését függetlenül attól, hogy a generatív modell milyen nyelvtani szerkezetet vagy rokon értelmű szavakat használt az elkészítési lépések leírásakor.

2.4.3. Kreativitás és Hallucináció mérése

A generatív képességek legmélyebb vizsgálatát egy egyedi, "kreatív maszkoláson" alapuló szkriptrészlet végezte. A tesztelés során az algoritmus szándékosan hiányos bemenetet adott a modelleknek. Ezt követően a szkript kinyerte a modell által generált hozzávalókat, és részleges sztring-egyezés vizsgálattal vetette össze azokat a valódi, elrejtett cél-hozzávalókkal. A visszahívás (recall) metrika azt mutatta meg, hogy a modell mennyire volt képes pusztán logikai alapon kitalálni és "visszaírni" a hiányzó összetevőket. Ezt egészítette ki a precízió (precision) mérése, amely a káros hallucinációk arányát, vagyis a bemenetből nem következő, logikátlan hozzávalók hozzáadását büntette.

3. Szoftvertechnológia és Webes Architektúra

3.1. Kliensoldali felület és renderelés

A felhasználói felület implementálásához a legújabb, React 19-es keretrendszert választottam Vite környezetben. A modern kliensoldali alkalmazások egyik legnagyobb kihívása a teljesítmény optimalizálása és a keresőmotorok (SEO) általi indexálhatóság. Ennek áthidalására szoftvermérnöki döntés volt a keretrendszer beépített szerveroldali renderelési (SSR - Server-Side Rendering) funkciójának alkalmazása a React Router v7 útválasztó bevonásával. Ez a megközelítés a hagyományos, kizárólag a böngészőben felépülő egyoldalas alkalmazásokkal (SPA) szemben biztosítja a HTML tartalom szerveren történő előgenerálását. Ez javítja az oldal kezdeti betöltési idejét (Largest Contentful Paint), miközben a kezdeti hálózati válasz megérkezése után a böngészőben végbemenő hidratációs (Hydration) folyamat révén az alkalmazás megőrzi a megszokott, azonnali reakciót biztosító interaktív élményt. A reszponzív, modern arculat kialakításához Tailwind CSS technológiát alkalmaztam, amely utility-first megközelítésével feleslegessé tette a globális CSS fájlok karbantartását, jelentősen felgyorsítva a komponensek stílusozását.

3.2. Szerveroldali architektúra (Backend) és Adatbázis

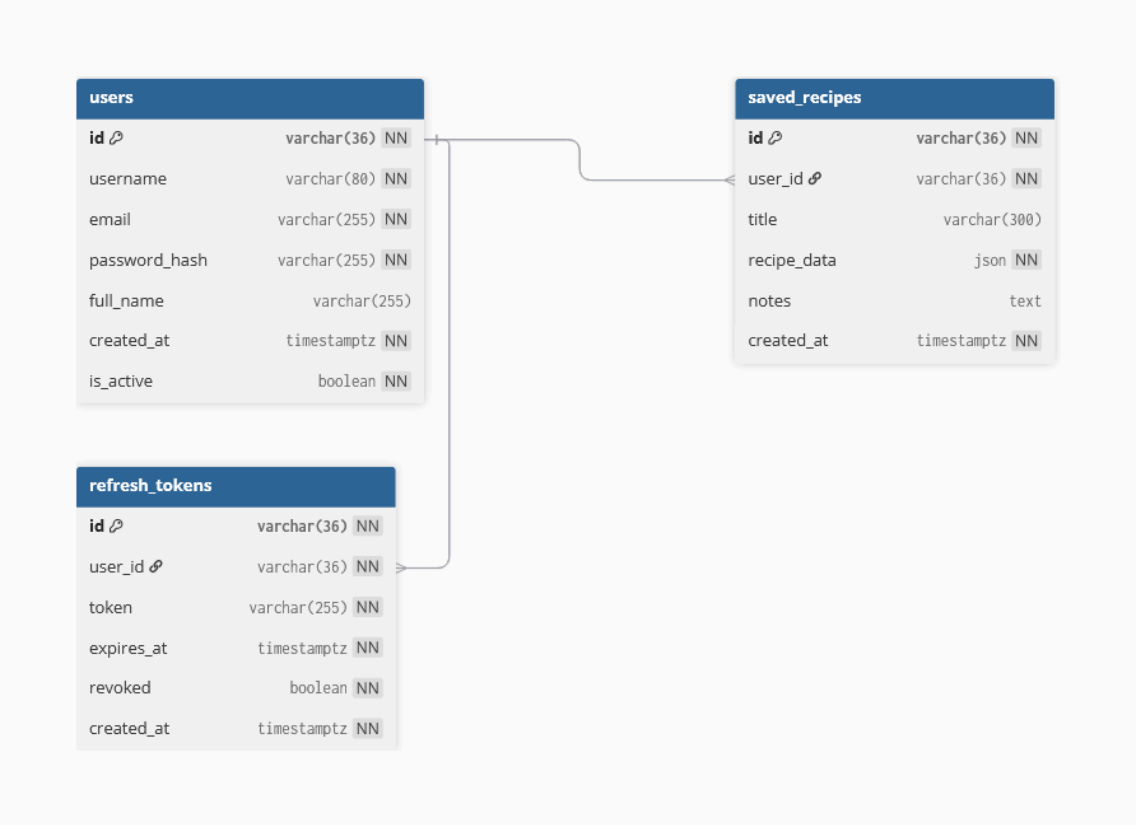
Az üzleti logika, a felhasználókezelés és a gépi tanulási modellek integrációjának kiszolgálására egy aszinkron Python backendet építettem fel FastAPI keretrendszerben. A FastAPI aszinkron (ASGI) működése elengedhetetlen volt a projekt szempontjából, mivel a külső nagy nyelvi modellek (például Gemini) API hívásai, valamint a nagyméretű saját Transformer modellek tenzorműveletei blokkoló folyamatok lennének egy hagyományos, szinkron szerveren. Az adatok perzisztens tárolását (felhasználói fiókok, hitelesítő adatok és mentett receptek) egy relációs PostgreSQL adatbázis látja el, amelyhez az alkalmazás SQLAlchemy ORM (Object-Relational Mapping) rétegen keresztül kapcsolódik, garantálva az SQL-injekciók elleni védelmet és a tranzakciók biztonságát.

3.3. Adatbázis-tervezés és Biztonságos Adatáramlás

A webalkalmazás működésének és a felhasználói adatok védelmének alapját a gondosan megtervezett relációs adatmodell és a biztonságos hálózati kommunikáció adja. A rendszer perzisztens rétegét egy PostgreSQL adatbázis biztosítja, amelynek sémáját a hatékonyság és a skálázhatóság figyelembevételével alakítottam ki.

3.3.1. Entitás-Kapcsolat (ER) modell és Adatszerkezet

A rendszer adatbázis-architektúrája három fő entitásra épül, amelyek szigorú idegenkulcs-kapcsolatokkal (Foreign Key) kapcsolódnak egymáshoz.



3.1. ábra: A rendszer egyed-kapcsolat diagramja, bemutatva a felhasználók, a tokenek és a receptek relációit.

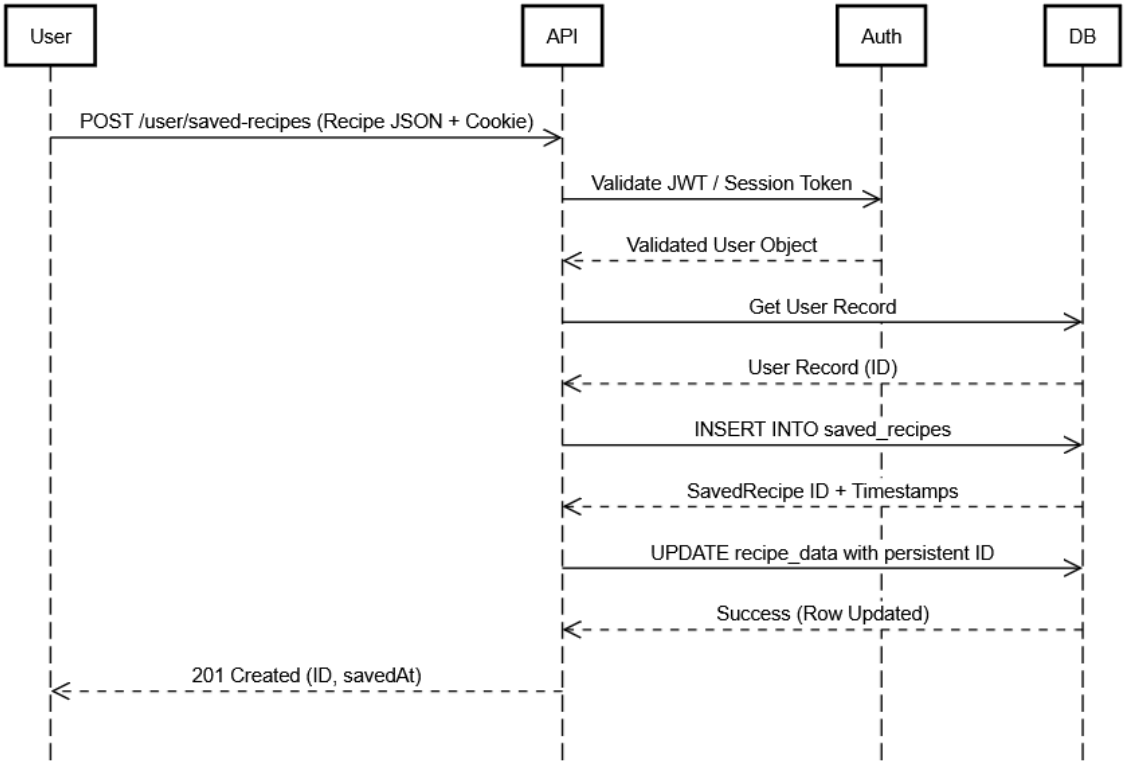
Ahogy a 3.1. ábrán látható, az adatmodell központi eleme a users tábla, amely a felhasználói fiókok alapvető azonosítóit tárolja. A hagyományos, növekvő sorszámok helyett a rendszer biztonsági okokból UUID (Universally Unique Identifier, varchar(36)) alapú elsődleges kulcsokat (id) használ, megelőzve az adatok felsorolásos sebezhetőségét (Insecure Direct Object Reference). A jelszavak tárolása nyílt szöveg helyett, sózott Argon2 algoritmussal hasított formában (password_hash) történik.

A munkamenetek (Session) biztonságos és állapotmentes (Stateless) kezelését a refresh_tokens tábla biztosítja. A rövid élettartamú JWT (JSON Web Token) hozzáférési tokenek mellett a rendszer hosszú távú frissítési tokeneket állít ki, amelyeket adatbázis-szinten tart nyilván. Ez a megoldás lehetővé teszi a kompromittálódott munkamenetek azonnali, szerveroldali visszavonását (a revoked logikai mező segítségével).

A generált receptek perzisztálása a saved_recipes táblában történik, amely egy egy-a-többhöz (1:N) relációval kapcsolódik a felhasználókhoz. Szoftvermérnöki döntés volt a PostgreSQL natív JSON adattípusának alkalmazása. Mivel a mesterséges intelligencia által generált receptek struktúrája (a hozzávalók és lépések száma) dinamikus, egy szigorú relációs táblafelbontás (például külön tábla a hozzávalóknak és külön a lépéseknek) indokolatlanul lelassította volna a lekérdezéseket. A JSON oszlop használata ötvözi a relációs adatbázisok adatintegritását a NoSQL adatbázisok rugalmasságával, ideális megoldást nyújtva az AI kimenetek tárolására.

3.3.2. Szekvenciális adatáramlás: A receptmentés folyamata

A kliens és a szerver közötti kommunikáció aszinkron REST API hívásokon keresztül valósul meg. Ennek egyik legkritikusabb és legösszetettebb folyamata a generált recept elmentése a felhasználó személyes profiljába, amelyet az alábbi szekvenciadiagram szemléltet.



3.2. ábra: A receptmentés (POST /user/saved-recipes) folyamatának szekvenciadiagramja.

A 3.2. ábra azokat az elosztott lépéseket szemlélteti, amelyek egy sikeres receptmentési művelet során zajlanak. A folyamat a kliensoldalon indul, ahol a

felhasználó kezdeményezi a mentést. Ennek hatására a böngésző egy POST kérést küld a /user/saved-recipes végpontra, amelynek törzsében a recept JSON formátumban kerül továbbításra, míg a fejlécek tartalmazzák a hitelesítéshez szükséges tokent.

A kérés beérkezése után a FastAPI alapú backend első lépésben hitelesítési ellenőrzést végez. Ennek során az Auth modul validálja a JSON Web Token aláírását és lejárati idejét. Amennyiben a token érvénytelen, a rendszer a folyamatot megszakítja, és HTTP 401 státuszkóddal tér vissza.

Sikeres hitelesítés esetén a backend lekéri a felhasználóhoz tartozó egyedi azonosítót, majd végrehajtja a szükséges adatbázis-műveletet, amely során a recept adatai beszúrásra kerülnek a saved_recipes táblába. A művelet befejeztével az adatbázis visszaadja az új rekord azonosítóját és a hozzá tartozó időbélyegeket.

Ezt követően a backend kiegészíti a memóriában tárolt recept objektumot az új azonosítóval, majd egy HTTP 201 (Created) státuszkódú válasszal tér vissza a klienshez. A válasz tartalma lehetővé teszi a frontend számára a felület frissítését és a mentési művelet sikeres végrehajtásának visszajelzését.

3.4. Fejlesztési és üzemeltetési környezet

A végleges, moduláris architektúra internetes környezetbe történő publikálásához a Render felhőszolgáltató platformot alkalmaztam. A rendszer platformfüggetlenségét Docker konténerizációval biztosítottam. A projektben megírt Dockerfile konfigurációk lehetővé tették a verziókövető rendszerhez (GitHub) kapcsolt automatikus integrációs és szállítási (CI/CD) folyamatok kialakítását: minden új kódfeltöltés automatikusan elindítja a buildelési folyamatot és frissíti az éles szervert. Bár az alkalmazás jelenleg egy kutatási demonstrációt szolgál a szolgáltató ingyenes erőforrásain, a Docker-alapú architektúra garantálja, hogy a rendszer a jövőben bármikor, kódmódosítás nélkül átemelhető és vertikálisan skálázható legyen egy ipari szerverkörnyezetbe.

4. Eredmények

A jelen fejezet bemutatja a teljes kutatási és fejlesztési folyamat végeredményét. Elsőként a publikált, interaktív webalkalmazás felhasználói felületét és az ahhoz kapcsolódó UX döntéseket részletezem, majd a háttérben futó gépi tanulási modellek teljesítményének objektív, metrikákon alapuló kiértékelését mutatom be.

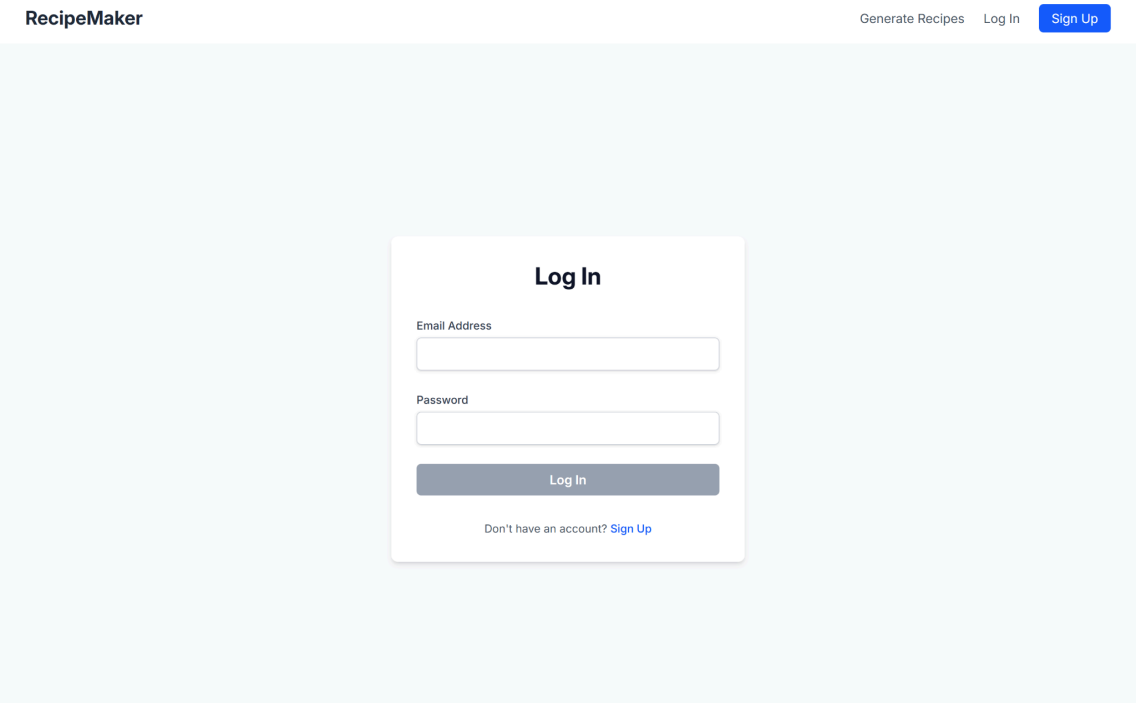
4.1. A webalkalmazás funkciói és felhasználói felülete

A szoftverfejlesztés kezdeti szakaszában a webalkalmazás ergonómiájának megtervezése drótvázak (wireframes) elkészítésével indult. A wireframek a mellékletek között megtalálhatóak.

Mivel a szoftver egy iteratív, agilis fejlesztési módszertanra épült, a kódolás során a felület folyamatosan fejlődött. A végleges felület kialakításánál a fő szempont a kognitív terhelés csökkentése és a mesterséges intelligencia aszinkron jellegének zökkenőmentes vizuális áthidalása volt.

4.1.1. Hitelesítés és kezdőképernyő

Az alkalmazás belépési pontja egy bejelentkezési és regisztrációs felület (Login/Register). A biztonságos hozzáférés érdekében a backend JWT (JSON Web Token) alapú hitelesítést használ, a jelszavakat pedig Argon2 algoritmussal hasítva (hashing) tárolja az adatbázisban. Sikeres hitelesítés után a React Router átirányítja a felhasználót a védett főoldalra, ahol a generáló űrlap kapott helyet.

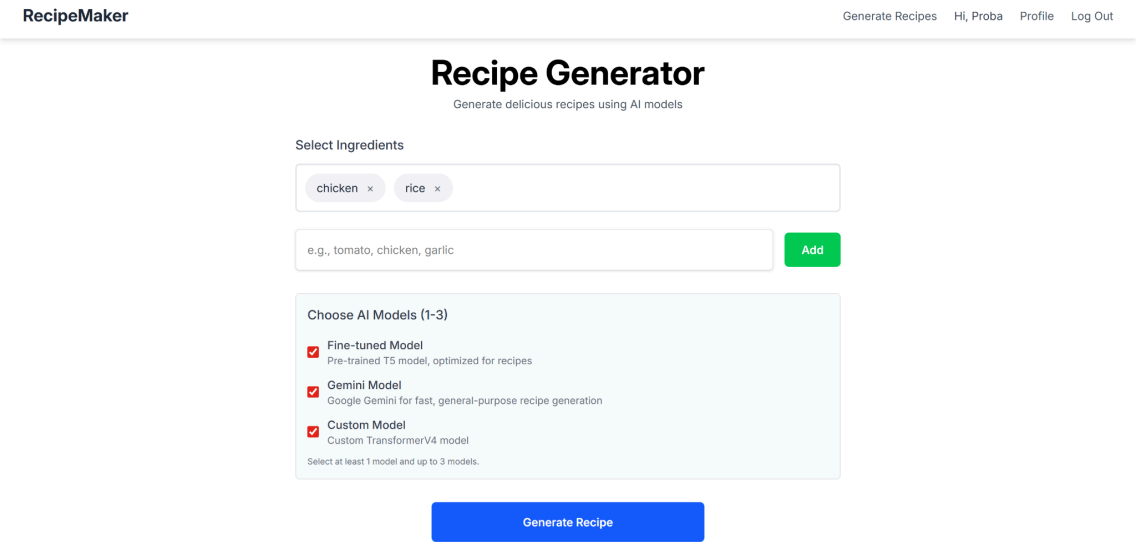


4.1. ábra: A receptgeneráló oldal bejelentkezési felülete

4.1.2. Dinamikus adatbevitel és Modellválasztás

A drótvázakon szereplő korai koncepcióhoz képest az egyik legfontosabb felületi módosítás a hozzávalók bevitelének áttervezése volt. A hagyományos, vesszőkkel elválasztott egyszerű szövegmező helyett egy dinamikus, vizuális címkéket (tag) kezelő rendszert implementáltam. A felhasználó által begépelt alapanyagok az Enter billentyű lenyomására különálló, színkódolt címkeként adódnak a felülethez, amelyek egy kattintással utólag is törölhetők. Ez a React állapotkezelésre (useState) épülő megoldás nem csupán az ergonómiát javítja, de szoftvermérnöki szempontból is kritikus: megakadályozza a formázási hibákat (pl. dupla vesszők, sortörések), így a backend már egy tiszta, feldolgozott tömböt kap.

Az adatbeviteli mező alatt a felhasználó jelölőnégyzetek segítségével párhuzamosan több modellt is kiválaszthat (Scratch Transformer, Finetuned T5, Gemini API), amivel egyedi A/B tesztelési lehetőséget kap a generált eredmények összehasonlítására.



4.2. ábra: Az app receptgeneráló felülete

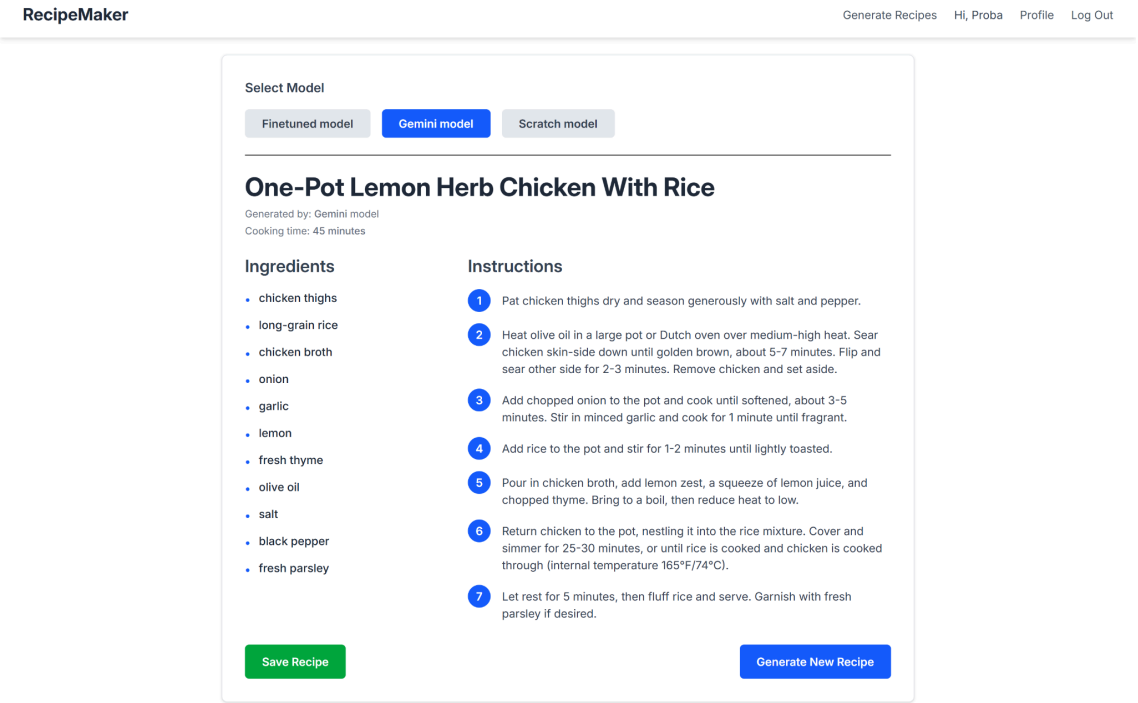
4.1.3. Aszinkron generálás és Optimista Állapotkezelés

A mesterséges intelligencia modellek futtatása és a Cloud API hívások több másodperces várakozási időt is igénybe vehetnek. Ennek kezelésére az alkalmazás nem navigál át egy különálló, statikus töltőképernyőre, hanem az úgynevezett "Optimista UI" elveit követve magába az indítógombba integrálja az állapotjelzést. Kattintáskor a gomb letiltódik, a felirata dinamikusan „Generating...” állapotra vált. Ez megakadályozza a többszörös beküldést, és a kontextus elvesztése nélkül ad vizuális visszajelzést a háttérfolyamatok futásáról.

4.1.4. Eredmények összehasonlítása és Perzisztencia

Miután a kiszolgáló megkapta az LLM modellek aszinkron válaszait, a felhasználó a generált receptet bemutató oldalra kerül. Mivel a szoftver támogatja a párhuzamos generálást, az eredményoldal egy füles (Tab) navigációs elrendezést kapott. Ez az architektúra lehetővé teszi, hogy a hálózati sávszélességet kímélve, az oldal újratöltése

nélkül lehessen váltogatni a különböző modellek által generált receptek között.



4.3. ábra: A generált receptek füles (tab) elrendezésű összehasonlító nézete.

Ezen a felületen a felhasználónak lehetősége van a tetsző recepteket egyetlen kattintással elmenteni az adatbázisba. Ezek az entitások a dedikált Profil oldalon gyűlnek össze. A mentett receptek listájának megnyitásakor a rendszer az eredeti generálási nézetet tölti be, egy kibővített funkciógombbal kiegészítve, amely lehetővé teszi a recept eltávolítását a gyűjteményből. A törlési művelet a backend oldalon kaszkádolt adatbázis-törléssel biztosítja az adatintegritást, megakadályozva az árva rekordok felhalmozódását a PostgreSQL adatbázisban.

4.2. A gépi tanulási modellek kiértékelése

A betanított modellek teljesítményének objektív összehasonlítását egy független, a tanítás során sosem látott teszthalmazon hajtottam végre. A kvantitatív (metrikákon alapuló) elemzés mellett kiemelt figyelmet fordítottam a generált kimenetek kvalitatív (minőségi és logikai) vizsgálatára is.

4.2.1. A saját fejlesztésű modell eredményei

Az alapoktól épített, aszimmetrikus Transformer hálózat esetében három különböző iterációt (V2, V3 és V4) értékeltem ki. Ennek a fázisnak a legfőbb kutatási kérdése az volt, hogy egy nulláról tanított hálózat hogyan reagál a különböző adatelőkészítési (Data Augmentation) stratégiákra.

A V2-es iteráció tanítása során a modell zajmentes adatokat kapott: a bemeneti prompt pontosan azokat a hozzávalókat tartalmazta, amelyeket a célérték (target) is elvárt. Ezzel szemben a V3-as és V4-es iterációk betanításakor egy tudatos zaj-injektálási stratégiát alkalmaztam: a bemeneti listából véletlenszerűen eltávolítottam releváns, illetve hozzáadtam irreleváns alapanyagokat. A mérnöki cél az volt, hogy a hálózatot kreativitásra, a logikátlan elemek szűrésére és az alapvető konyhai hozzávalók önálló kiegészítésére kényszerítsem.

Metrika	**V2**	**V3**	**V4**
Formátumkövetés	100.00%	100.00%	99.00%
ROUGE-1	53.39%	21.93%	22.12%
BERTScore	89.20%	83.06%	82.07%
Precision	96.14%	17.36%	25.94%
Recall	99.47%	7.11%	11.93%

4.4. táblázat: a saját fejlesztésű modell metrikái

A táblázat adatai jelentős különbségeket, valamint egy fontos jelenséget mutatnak. A zaj nélküli adatokon tanított V2-es iteráció magas értékeket ért el (99.47% visszahívás, 96.14% precizitás). Ennek hátterében a hálózat aszimmetrikus kódoló rétegeiben kialakult másolási mechanizmus áll. A modell megtanulta, hogy a bemeneti promptban szereplő hozzávalókat változtatás nélkül emelje át a generált struktúrába. Bár ez a viselkedés a metrikák szintjén kedvező eredményeket produkál, a modell rugalmatlan marad, és nem képes érdemi kulináris kiegészítések előállítására.

A kreativitás kikényszerítése céljából zajjal terhelt adatokon tanított V3-as és V4-es modellek eredményei azonban rávilágítanak a nulláról történő tanítás korlátaira.

A bemenet megbízhatatlanná válása miatt a hálózatok figyelmi-mechanizmusa (Self-Attention) nem tudta elsajátítani a mélyebb kulináris logikát. Ehelyett fellépett a szakirodalomban "Mode Collapse" (állapot-összeomlás) néven ismert jelenség: a modell megtanulta szinte teljesen ignorálni a bemeneti promptot, és a tanítóhalmazból memorizált, kontextuson kívüli recepteket kezdett el hallucinálni (ez magyarázza a V3-as modell 7.11%-os Recall és 17.36%-os Precizitás értékét). A V4-es architektúra a fejlett dekódolási stratégiáknak és a regularizációnak köszönhetően kismértékben tompította ezt a hibát, de az alapvető problémát nem oldotta meg.

4.2.2. A finomhangolt Flan-T5 modell eredményei

A transzfertanulás hatékonyságának demonstrálására a Flan-T5 modell három különböző finomhangolási iterációját (V3, V4, valamint egy emberibb dekódolású V5 verziót) értékeltem ki.

Metrika	Flan-T5 (V3)	Flan-T5 (V4)	Flan-T5 (V5)
Formátumkövetés	85.00%	100.00%	100.00%
ROUGE-1	24.62%	46.26%	44.52%
BERTScore	81.36%	88.70%	88.31%
Precision	23.05%	98.14%	96.81%
Recall	26.35%	83.80%	79.66%

4.5. táblázat: a finomhangolt modell metrikái

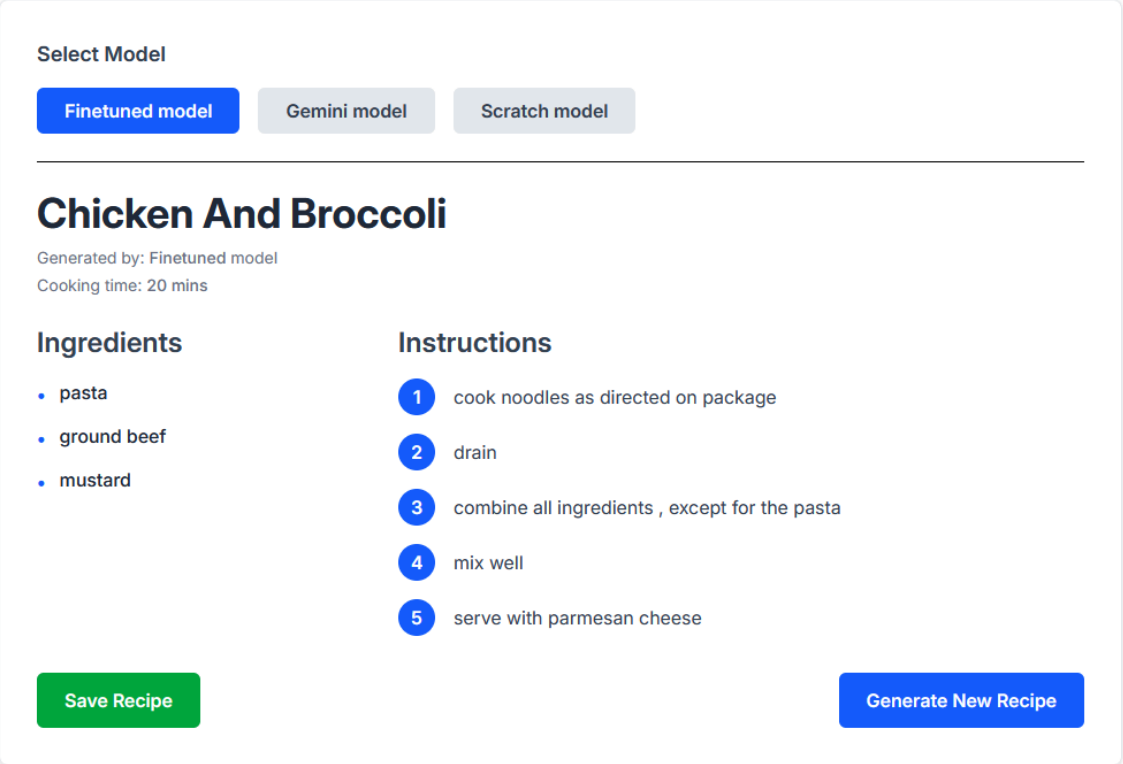
A táblázat jól mutatja a hiperparaméter-optimalizálás sikerét: míg a korai V3-as T5 iteráció a strukturális (85%) és logikai (23.05% Precizitás) feladatokban is alulmúlta az elvárásokat, a végső V4-es modell javította az eredményeken. A V4 100%-os formátumkövetést, 88.70%-os BERTScore-t és 98.14%-os Precizitást ért el, szinte teljesen kiküszöbölve a káros hallucinációkat.

A kutatás utolsó fázisában (V5) vizsgált kreatív dekódolási stratégia (magasabb hőmérsékletű mintavételezés - Sampling) igazolta a szakirodalomban leírt elméleteket. Bár a szándék az emberibb és változatosabb szöveggenerálás volt, a kvalitatív vizsgálat megerősítette a metrikák (csökkenő Precizitás és Recall) által jelzett tendenciát: a V5-ös

modell által generált receptek kulináris logikája és érthetősége elmaradt a V4-es verziótól. A kreativitás növelése gyakran vezetett nyelvi inkoherenciához és kevésbé praktikus utasításokhoz. Ez a kompromisszum rávilágít arra, hogy egy receptgeneráló szoftvernél a determinisztikus, szigorú szabálykövetés (V4) a gyakorlatban értékesebb, mint az asszociatív, szabadabb "kreativitás" (V5).

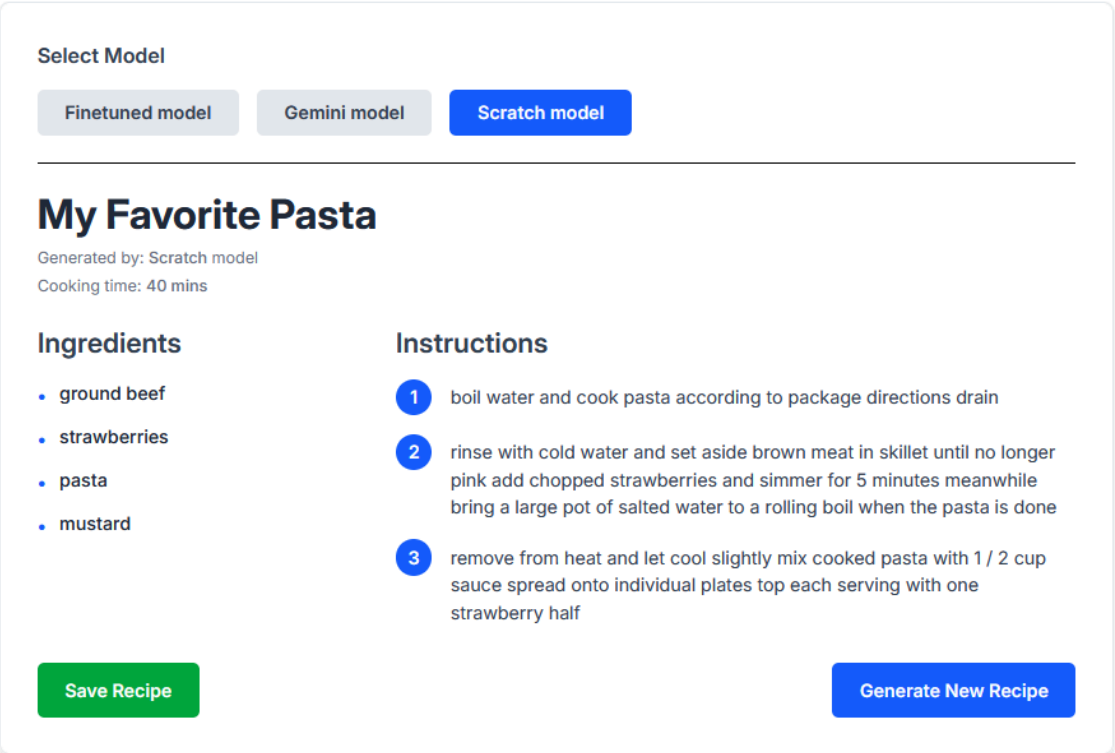
4.2.3. Kvalitatív összehasonlítás

A statisztikai metrikák eredményeit a modellek generált kimeneteinek kvalitatív vizsgálata is alátámasztja. A teszt során a modelleknek négy, egymással kulinárisan nehezen összeegyeztethető hozzávalóból (darált marhahús, eper, tészta, mustár) kellett strukturált receptet alkotniuk a fejlesztett webalkalmazás felületén.



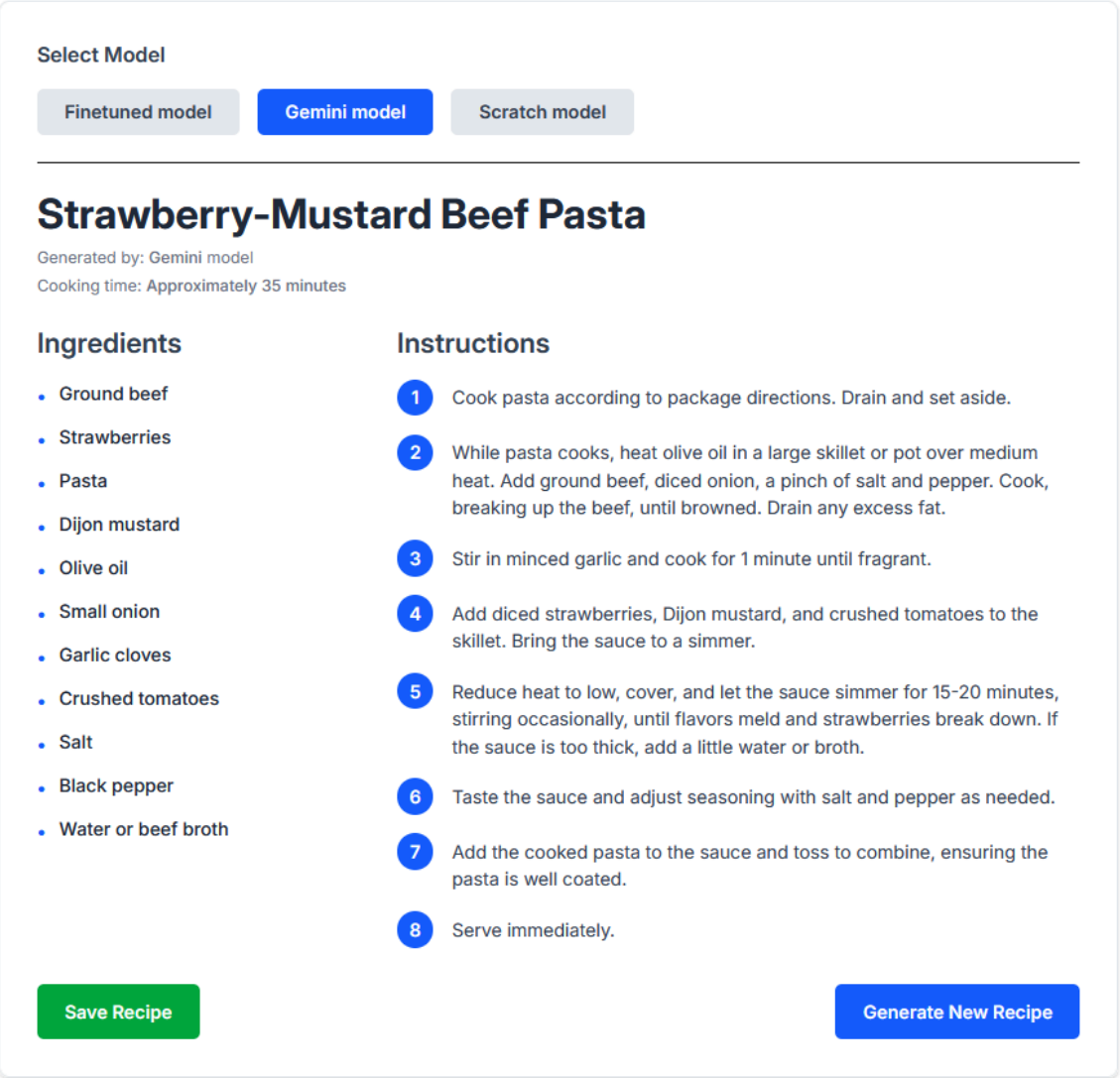
4.6. ábra: A finomhangolt Flan-T5 modell eredménye a zajos bemenetre.

A 4.6. ábra a finomhangolt hálózatnál fellépő memóriatorzítást szemlélteti. A modell a szokatlan bemenet hatására elvesztette a kontextust: a recept címét a bemenettől független "Chicken And Broccoli" kifejezésre generálta, az epret pedig ignorálta az elkészítési folyamatból. A kimenet egy sablonos, a tanítóhalmaz eloszlását tükröző sajtos tészta receptje lett, ami igazolja a kisméretű hálózat rugalmatlanságát az ismeretlen adat-kombinációkkal szemben.



4.7. ábra: A saját fejlesztésű (Scratch) modell eredménye a zajos bemenetre.

A 4.7. ábrán látható nulláról tanított modell betartotta a formai követelményeket és az összes megadott hozzávalót felhasználta, azonban a kimenet logikailag hibás. A darált hús eperrel történő összefőzése rámutat arra, hogy a modell pusztán statisztikai alapon, a tokenek együttes előfordulási valószínűsége alapján illeszti egymás mellé a szavakat, de nem rendelkezik a fizikai világ szabályainak (például konyhai eljárások és ízharmóniák) ismeretével.



4.8. ábra: A Google Gemini API eredménye a zajos bemenetre.

A 4.8. ábra a szolgáltatásként igénybe vett (LLM-as-a-Service) Gemini modell kimenetét mutatja. A modell a rendszerpromptban engedélyezett mozgásteret kihasználva releváns kiegészítő alapanyagokat (hagyma, fokhagyma, olívaolaj, paradicsompüré, alaplé) adott a recepthez. A generálás kulináris koherenciáját az biztosítja, hogy az architektúra az epret és a mustárt egy savas-édes szószalapként kezelte, amelyet a többi hozzávalóval összefőzve logikailag helyes utasítássort alkotott. Ez a kimenet megerősíti, hogy a komplex, asszociatív feladatok végrehajtásához a nagy paraméterszámú modellek masszív előtanításából származó lexikális kontextus elengedhetetlen. A bemutatott zajos példa mellett a modellek kiértékelése további két forgatókönyv alapján is megtörtént: egy hagyományos, logikus alapanyagokat tartalmazó "klasszikus" teszt, valamint egy csupán két összetevőből álló, a modellek kiegészítő képességét vizsgáló "minimalista" teszt formájában. Ezen kiegészítő tesztek

vizuális eredményei és összehasonlításai a dolgozat végén, az M.5. és M.6. mellékletben kaptak helyet.

4.2.4. Összehasonlító konklúzió

A kiértékelés eredményei alapján a szakdolgozat hipotézisét az eredmények alátámasztják. A nulláról tanított (Scratch) architektúra bár képes volt a másolási mechanizmus (V2) kialakítására, a kreativitást megkövetelő zajos adatokkal (V3, V4) már nem tudott megbirkózni; a bemenet ignorálásának (Mode Collapse) és a logikátlan hallucinációnak a csapdájába esett. Ezzel szemben az előtanított Flan-T5 (V4) modell a masszív, önfelügyelt tanulással megszerzett lexikális "világtudását" sikeresen adaptálta a feladathoz: zajos bemenet esetén is maximalizálta a Precizitást (98.14%) és a koherenciát. A fejlesztett webalkalmazás gyártási (production) környezetbe emeléséhez így a transzfertanulással optimalizált hálózat bizonyult az egyetlen megbízható megoldásnak.

5. Összefoglalás

A jelen szakdolgozat célja egy olyan mesterséges intelligenciára épülő webalkalmazás megtervezése, fejlesztése és kiértékelése volt, amely képes a felhasználó által megadott hozzávalókból strukturált, kulinárisan logikus recepteket generálni. A projekt azonban túlmutatott egy egyszerű szoftverfejlesztési feladaton: a munka gerincét egy átfogó gépi tanulási (Machine Learning) kutatás adta, amely a nulláról történő hálózatépítés és a transzfertanulás (Transfer Learning) paradigmáit állította szembe egy komplex, strukturált szöveggenerálási (NLP) feladatban.

A kutatás során megfogalmazott hipotézis szerint a transzfertanuláson alapuló, előtanított modellek alkalmazása hatékonyabb megoldást nyújt, mint a nulláról tanított neurális hálózatok egy komplex szöveggenerálási feladat esetén. Ezt a feltevést a kiértékelési eljárás eredményei támasztották alá, amelyek a ROUGE, BERTScore, precizitás és visszahívás metrikákon alapultak.

A saját fejlesztésű, PyTorch keretrendszerben implementált aszimmetrikus Transformer architektúra esettanulmányként szolgált a neurális modellek működésének vizsgálatához. Az eredmények azt mutatták, hogy egy nulláról tanított, 12 000 szavas szótárral rendelkező modell képes stabil másolási mechanizmus elsajátítására strukturált, zajmentes adatok esetén. Ugyanakkor zajjal terhelt bemeneteknél, ahol a feladat nagyobb fokú általánosítást és kreativitást igényel, a modell teljesítménye jelentősen romlott. Az ilyen környezetben megfigyelt állapot-összeomlás arra utal, hogy a kulináris logika és a nyelvi helyesség együttes modellezése meghaladja egy korlátozott kapacitású, nulláról tanított hálózat lehetőségeit.

Ezzel szemben az előtanított flan-t5-base modell finomhangolása számottevő javulást eredményezett minden vizsgált metrika esetében. A transzfertanulás során a modell a korábban elsajátított nyelvi reprezentációkat hatékonyan adaptálta a kulináris domén sajátosságaihoz. A finomhangolt modell a teszthalmazon teljes formátumkövetést mutatott, miközben magas BERTScore és precizitás értékeket ért el. Az eredmények alapján a modell képes volt csökkenteni a kontextuson kívüli vagy irreleváns kimenetek előfordulását is.

Ugyanakkor a kutatás egyik legfontosabb elméleti tanulsága a kvantitatív metrikák és a valós használhatóság közötti szakadék felismerése volt. Bár a magas pontszámok megmutatják, hogy a modell maradéktalanul elsajátította a receptírás

szintaktikai szabályait és a hozzávalók statisztikai együttállását (például megtanulta, hogy a liszthez gyakran tojás társul), a kvalitatív vizsgálat rávilágított a tisztán nyelvi modellek korlátaira. A generált receptek a gyakorlatban sok esetben továbbra sem eredményeztek volna fogyasztható vagy kulinárisan élvezhető ételeket, mivel a gép a statisztikai valószínűségeket modellezi, és nem érti a fizikai vagy kémiai konyhai folyamatokat. Ez az eredmény rávilágít arra, hogy a tisztán nyelvészeti alapú (LLM) megközelítés önmagában nem képes tökéletesen reprodukálni az emberi főzési intuíciót.

A kutatási eredmények gyakorlati integrációja egy moduláris architektúrában öltött testet. A frontend a legújabb React 19 technológiára és szerveroldali renderelésre épül, amely aszinkron, optimista állapotkezeléssel biztosítja a mesterséges intelligencia késleltetéseinek vizuális áthidalását. A backend egy Docker-konténerizált FastAPI rendszerként funkcionál, amely PostgreSQL adatbázissal és JWT hitelesítéssel védi a felhasználói adatokat. A rendszer és a külső felhőszolgáltatások (Google Gemini API) stabilitását egy exponenciális visszalépéses algoritmus garantálja.

5.1. Továbbfejlesztési lehetőségek

Bár az elkészült szoftver stabil és funkciókban gazdag prototípus, a felépített elosztott architektúra számos jövőbeli fejlesztési lehetőséget biztosít. A gépi tanulási vonalon érdemes lehet a jövőben a LoRA (Low-Rank Adaptation) technikák vizsgálata még nagyobb nyílt forráskódú modelleken (például Llama 3), hogy a lokális generálás minősége tovább javuljon. Szoftvermérnöki szempontból az alkalmazás kiegészíthető egy dinamikus tápanyagkalkulátorral, amely egy külső API (például USDA) segítségével automatikusan kiszámítja a generált recept kalória- és makrotápanyag-értékeit, tovább növelve a szoftver gyakorlati hasznát a tudatos táplálkozást követő felhasználók számára.

Összességében a szakdolgozat sikeresen megmutatta, hogy az elméleti adatközpontú AI-kutatás és a modern szoftvermérnöki elvek ötvözésével egy technológiailag előremutató alkalmazás hozható létre.

6. Mesterséges intelligencia használata a fejlesztés során

A szoftverfejlesztés az elmúlt években jelentősen átalakult a generatív mesterséges intelligencia térhódításával. A szakdolgozat elkészítése során a nagy nyelvi modellek és a különböző AI-alapú kódolási asszisztensek szerves részét képezték a munkafolyamatnak.

6.1. Alkalmazott eszközök és feladatkörök

A fejlesztés életciklusa során nem egyetlen modellt, hanem a feladat jellegéhez illeszkedő AI szolgáltatásokat alkalmaztam. A kódolási fázisban a GitHub Copilot biztosította a folyamatos integrációt közvetlenül a fejlesztői környezetben. A Copilot felületén keresztül dinamikusan váltogattam a beépített modellek között: használtam az Anthropic Claude Sonnet 4.5 modelljét, az OpenAI GPT-5.3-Codex modelljét, valamint a Google Gemini 3.1 Pro modelljét. Emellett a munkafolyamat kiegészült az Alibaba Qwen 3.5 modelljének parancssori (CLI) integrációjával, illetve a Gemini natív webes felületével.

A munkafolyamatokat szétválasztottam az eszközök erősségei alapján. Az architektúra-tervezéshez és a főbb döntéshozásokhoz, mint a technológiai stack kiválasztásához, a Google Gemini webes felületét használtam.

A repetitív kódok, adatosztályok, valamint a unit tesztek generálását jellemzően a Copilot környezetébe ágyazott Claude és Codex modellekre bíztam, valamint a Qwen CLI-t alkalmaztam.

A megírt és működő, de optimalizálásra szoruló kódblokkokat (például komplexebb adatbázis-lekérdezéseket) a Gemini és a Qwen segítségével vizsgáltattam felül.

A tudományos szakirodalom előzetes feldolgozásához, a cikkek összefoglalásához, valamint a forráskód kommenteléséhez és a dolgozat szövegének formálásához elsősorban a Geminit és a Qwent vettem igénybe.

6.2. Validációs módszertan és kritikus hibakezelés

A mesterséges intelligencia egyetlen kimenetét sem kezelem kész tényként.. A validációt többszintű módszertannal végeztem: a generált kódokat manuális

kódolvasásnak vetettem alá, majd egy másik AI modellel is ellenőriztettem a logikát. A végső minőségbiztosítást a megírt automatizált tesztek lefuttatása és a manuális végpont-tesztelés garantálta.

Ez a szigorú ellenőrzési folyamat elengedhetetlen volt, ugyanis a fejlesztés során több alkalommal is előfordult, hogy a modellek hibás, kontextusidegen vagy nem biztonságos megoldásokat javasoltak.

Jó példa erre a dokumentáció és a felületi kódolás fázisa: a Qwen modell a frontend dokumentációjának generálásakor hajlamos volt nem létező, "hallucinált" Tailwind CSS hexadecimális színkódokat generálni az érvényes utility osztályok helyett. Ezt a hibát vizuális ellenőrzéssel észleltem, majd a kódrészletet átadtam a Gemini modellnek, amely sikeresen kijavította a szintaxist a helyes keretrendszer-specifikus formátumra.

Egy másik, jóval kritikusabb szoftvermérnöki hiba a környezeti változók (.env) backend oldali kezelésénél lépett fel. Az egyik asszisztens egy olyan betöltési logikát generált a hitelesítési kulcsokhoz, amely lokális környezetben működött ugyan, de a Docker-alapú konténerizált környezetben nem találta meg a változókat. Mivel a modell nem látta át a teljes operációs rendszeri és szerver-infrastruktúrát, ezt a hibaforrást manuálisan kellett azonosítanom és kijavítanom.

6.3. Az emberi tényező: Területek AI-támogatás nélkül

A webalkalmazás felhasználói felületének vizuális tervezését, a drótvázak megalkotását és az ergonómiai kialakítást teljesen manuálisan végeztem. Szintén önálló, gépi beavatkozás nélküli folyamat volt a kísérleti eredmények és a metrikák szakmai elemzése és konklúzióinak levonása.

6.4. Tapasztalatok és jövőbeli tanulságok

Az AI eszközök használata jelentősen felgyorsította a sablonkódok megírását. Ugyanakkor a komplexebb generált függvények logikai átláthatósága gyakran alacsony volt, ami nehezítette a debuggolást. A legfontosabb tanulság a megfelelő kontextuskezelés: a jövőben izoláltabb kontextusblokkokkal tervezem irányítani a modelleket a megbízhatóság növelése érdekében.

Irodalomjegyzék

[1] I. Sutskever, O. Vinyals, és Q. V. Le, „Sequence to Sequence Learning with Neural Networks”, 2014. szeptember 10. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1409.3215

[2] A. Vaswani *és mtsai.*, „Attention Is All You Need”, 2017. június 12. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1706.03762

[3] C. Raffel *és mtsai.*, „Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer”, 2019. október 23. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1910.10683

[4] H. W. Chung *és mtsai.*, „Scaling Instruction-Finetuned Language Models”, 2022. október 20. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/2210.11416

[5] A. Holtzman, J. Buys, L. Du, M. Forbes, és Y. Choi, „The Curious Case of Neural Text Degeneration”, 2019. április 22. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1904.09751

[6] J. Howard és S. Ruder, „Universal Language Model Fine-tuning for Text Classification”, 2018. január 18. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1801.06146

[7] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries”, in *Text Summarization Branches Out*, 2004, o. 74–81.

[8] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, és Y. Artzi, „BERTScore: Evaluating Text Generation with BERT”, 2019. április 21. Elérés: 2026. április 21. [Online]. Elérhető: http://arxiv.org/abs/1904.09675

[9] Z. Ji *és mtsai.*, „Survey of Hallucination in Natural Language Generation”, febr. 2022, doi: 10.1145/3571730.

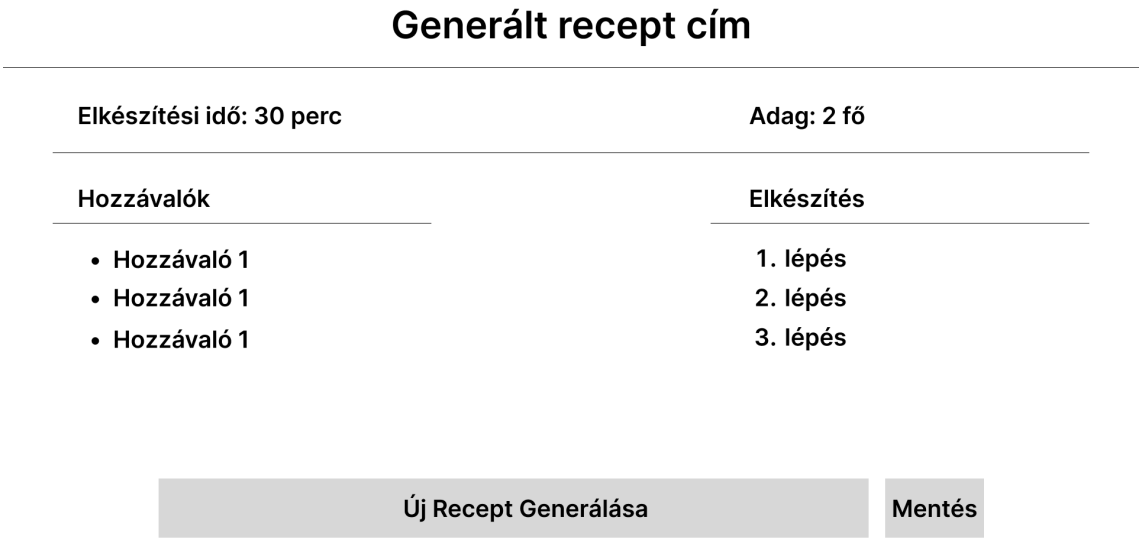
[10] S. Li, „Food.com Recipes and Interactions”. 2019. november 8. Elérés: 2026. április 21. [Online]. Elérhető: https://www.kaggle.com/shuyangli94/food-com-recipes-and-user-interactions

Köszönetnyilvánítás

Nyilatkozat

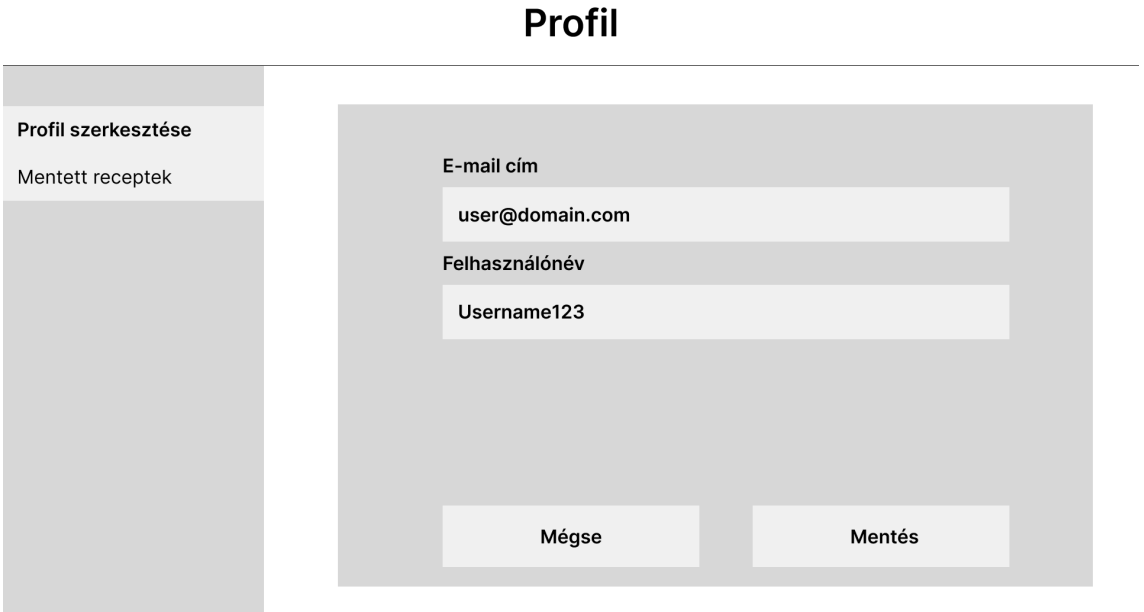
Mellékletek

M.1. ábra: A főoldal hálóterve

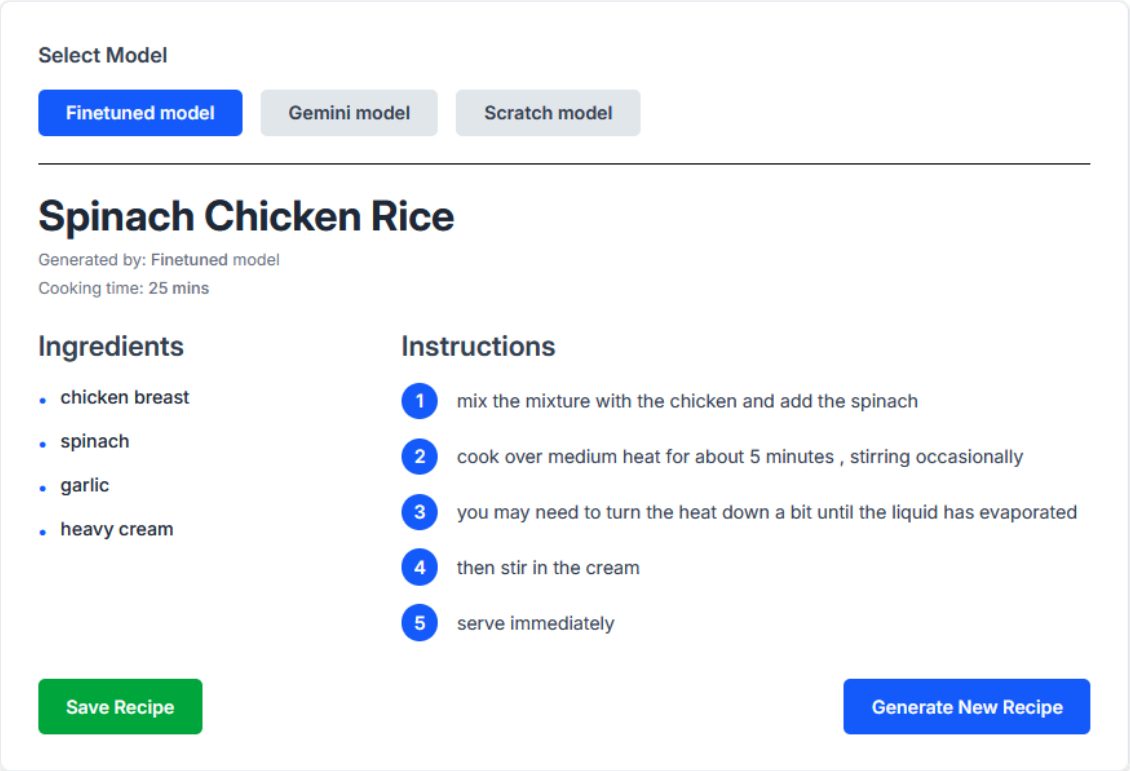


M.2. ábra: A generált recept oldal hálóterve

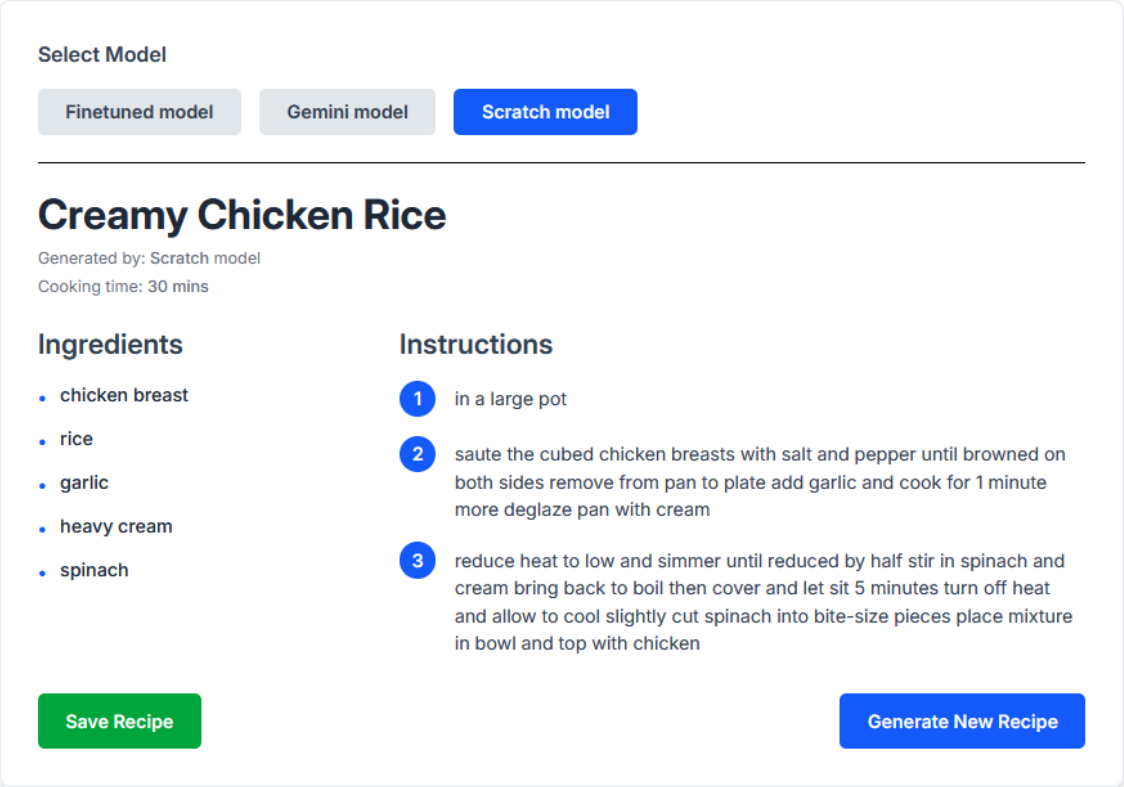
M.3. ábra: A profil mentett receptek oldalának hálóterve



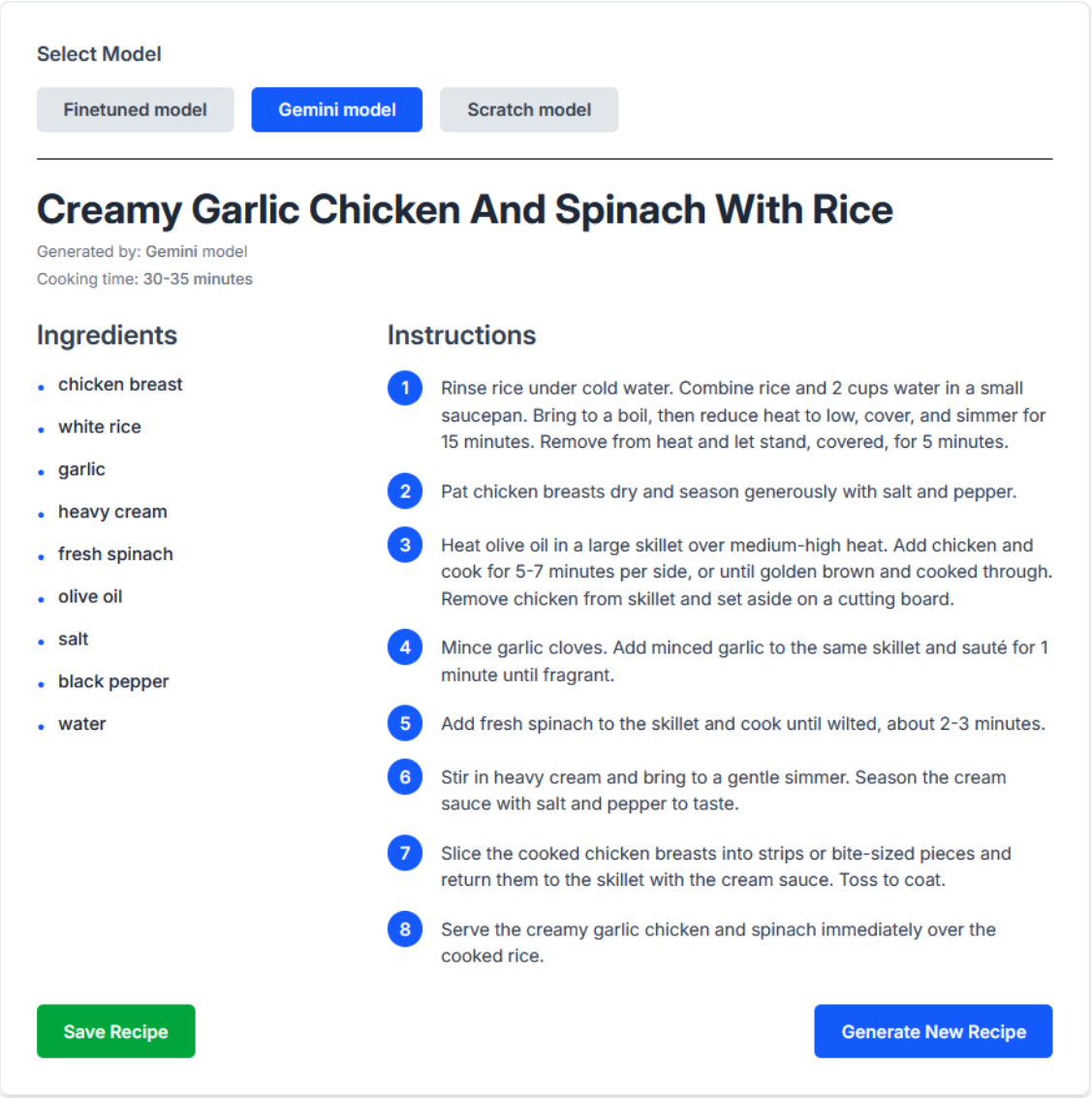
M.4. ábra: A profil szerkesztésének hálóterve



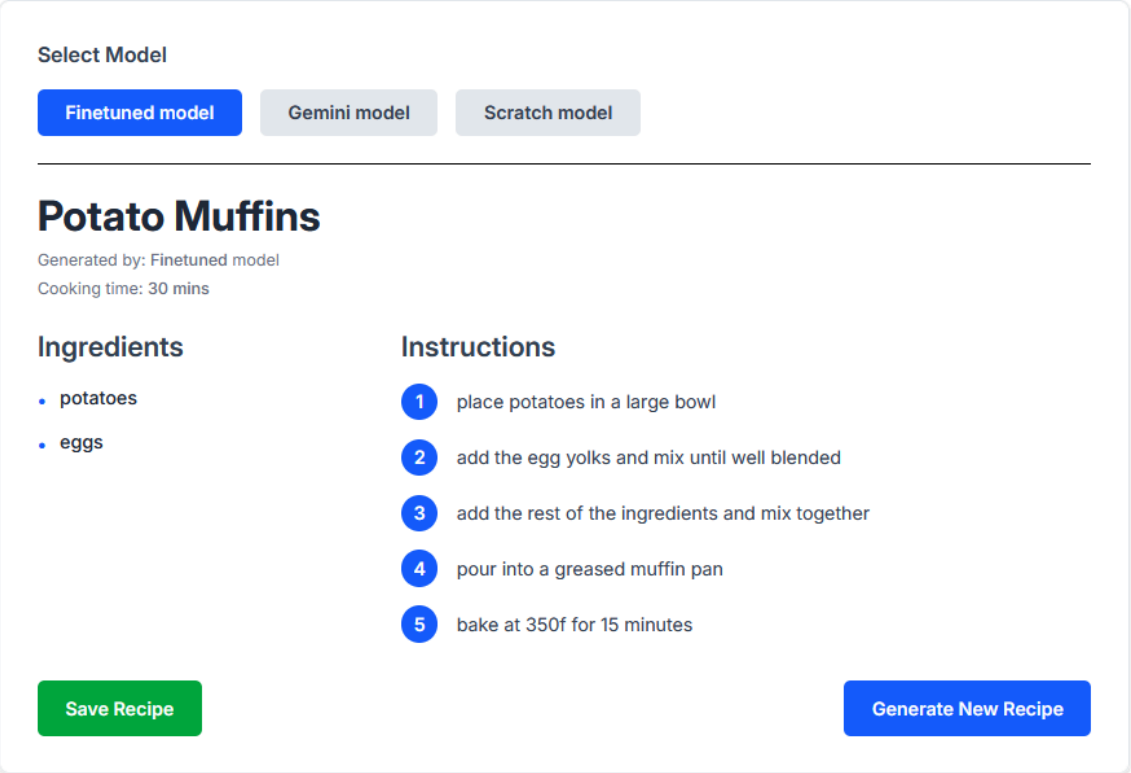
M. 5.1 ábra: A finomhangolt modell a klasszikus teszten



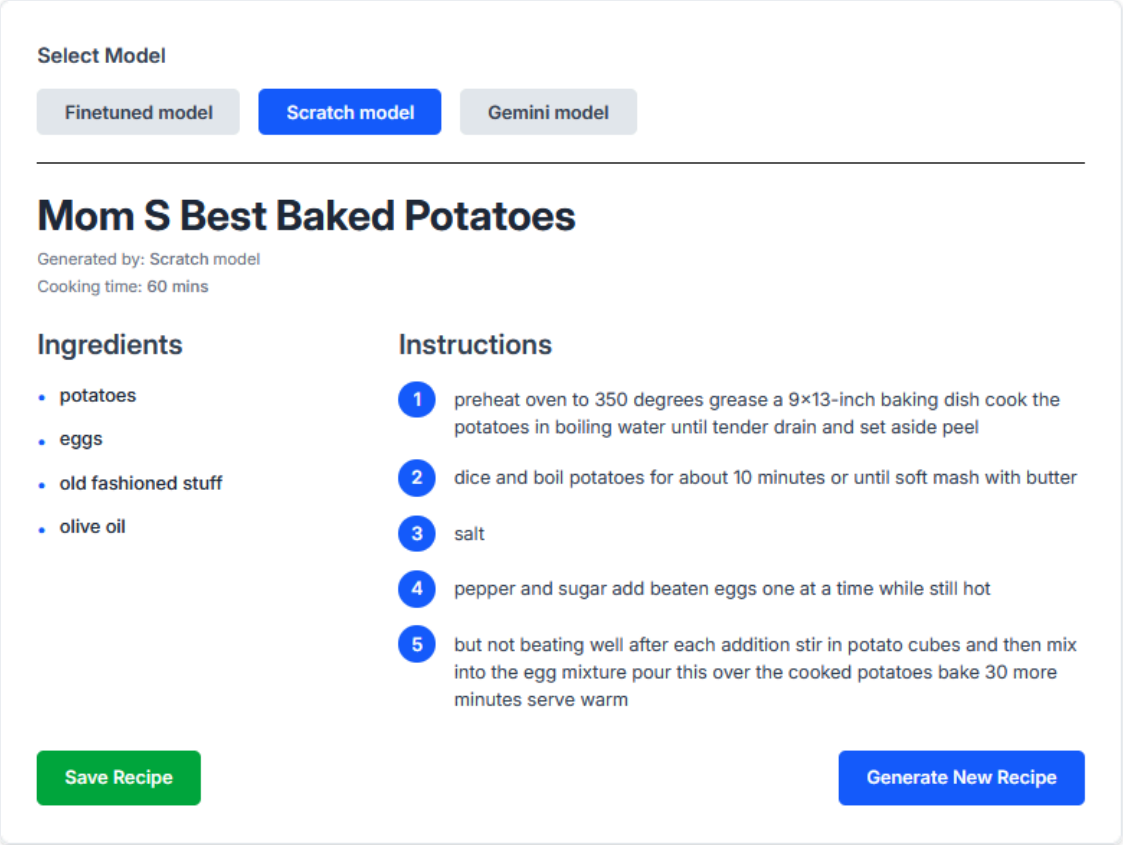
M. 5.2. ábra: A saját fejlesztésű modell a klasszikus teszten



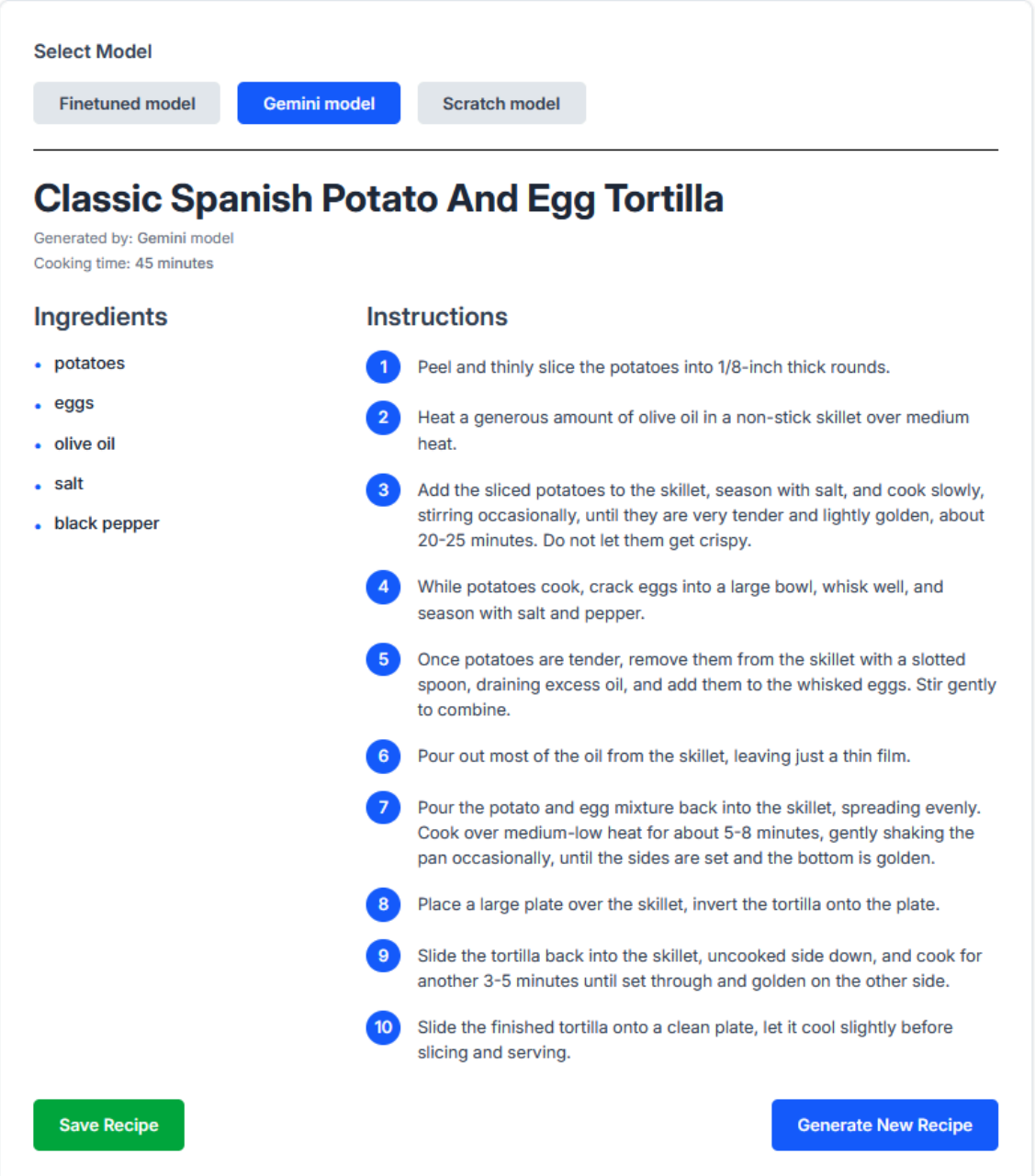
M 5.3. ábra: Google Gemini a klasszikus teszten



M. 6.1 ábra: A finomhangolt modell a minimalista teszten



M. 6.2. ábra: A saját fejlesztésű modell a minimalista teszten



M 6.3. ábra: Google Gemini a minimalista teszten